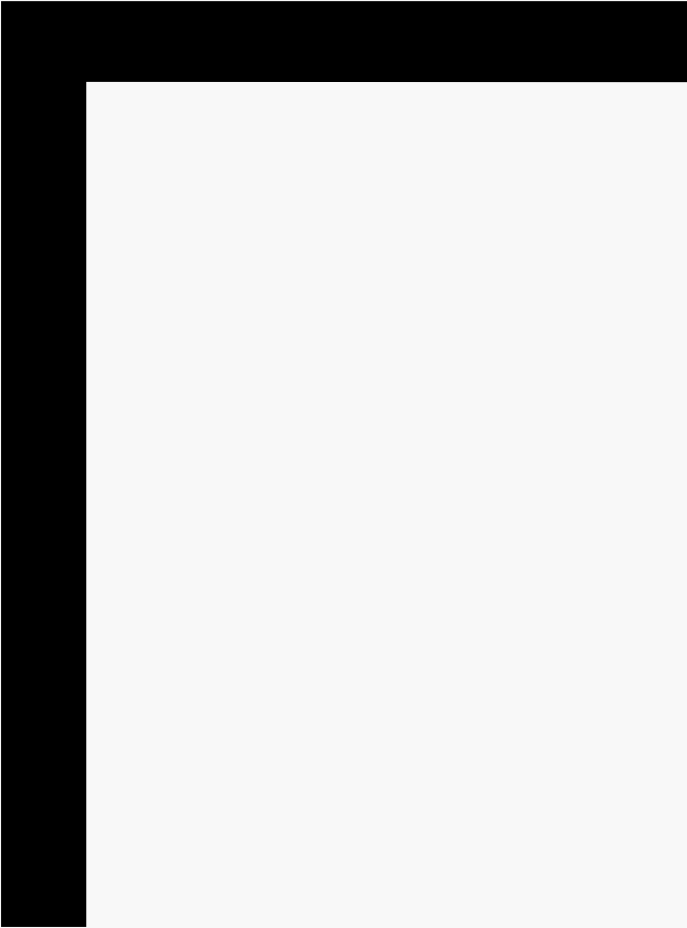
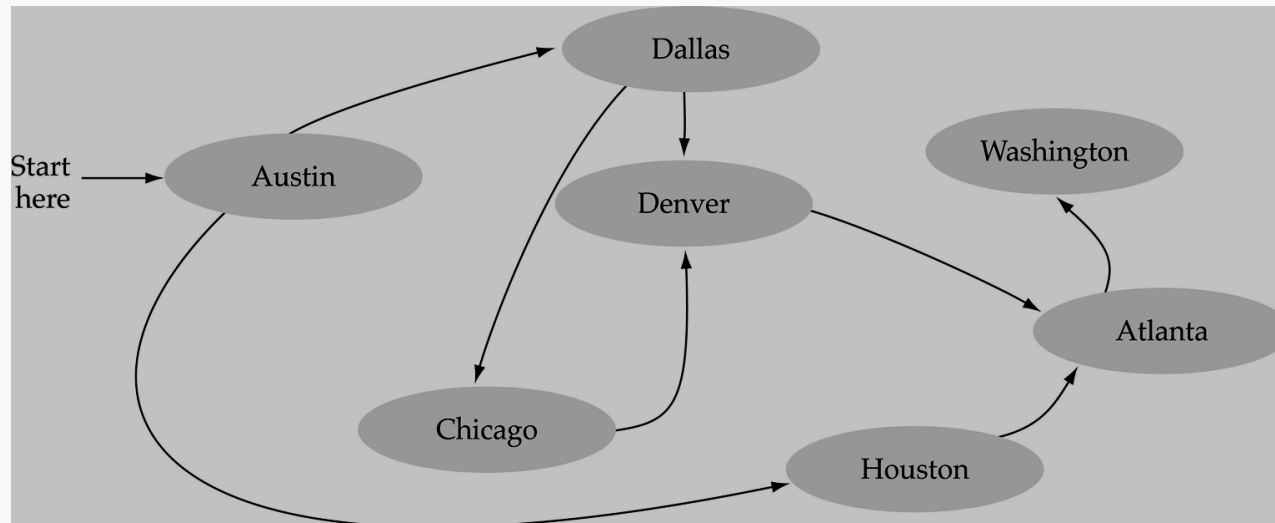




GRAPHS

What is a graph?

- A data structure that consists of a set of nodes (*vertices*) and a set of edges that relate the nodes to each other
- The set of edges describes relationships among the vertices



Formal definition of graphs

- A graph G is defined as follows:

$$G=(V,E)$$

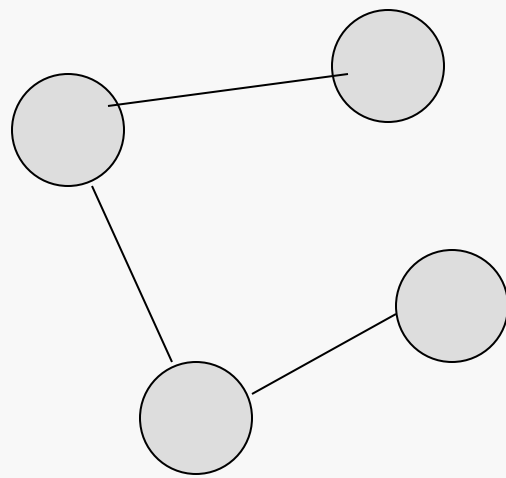
$V(G)$: a finite, nonempty set of vertices

$E(G)$: a set of edges (pairs of vertices)

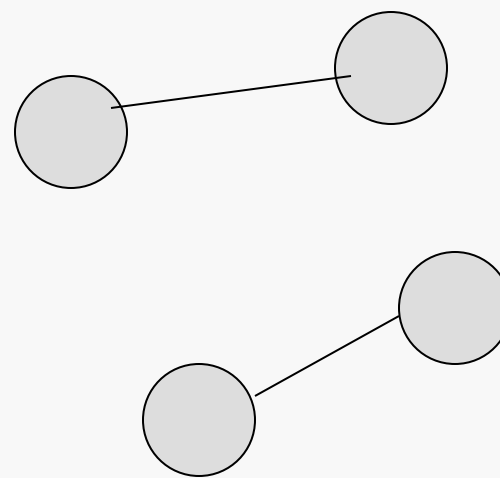
Introduction (Terminology)

- Vertices adjacent to a given vertex are called its *neighbors*.
- A *path* is a sequence of edges.
- A graph is said to be *connected* if there is at least one path from every vertex to every other vertex.
- A **non-connected** graph consists of several connected components.

Connected vs Non-connected graph



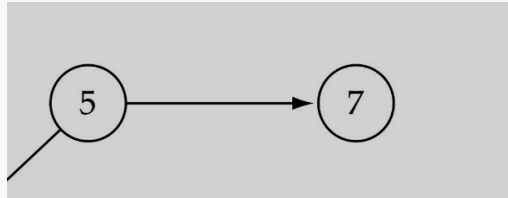
Connected graph



Non-Connected graph

Graph terminology

- Adjacent nodes: two nodes are adjacent if they are connected by an edge



- Path: a sequence of vertices that connect two nodes in a graph
- Complete graph: a graph in which every vertex is directly connected to every other vertex

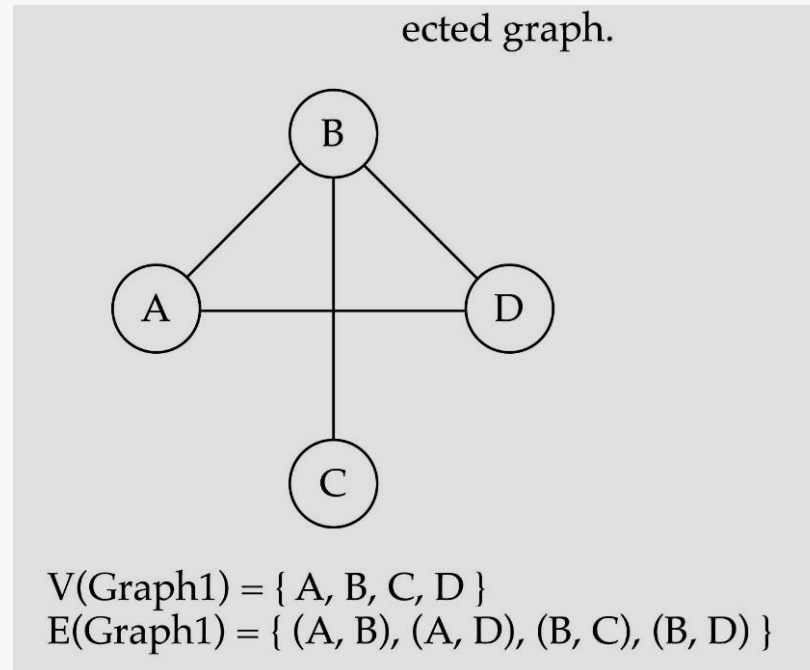
5 is adjacent to 7
by a directed edge from

Adjacent and Incident

- If (v_0, v_1) is an edge in an undirected graph,
 - v_0 and v_1 are **adjacent**
 - The edge (v_0, v_1) is incident on vertices v_0 and v_1
- If $\langle v_0, v_1 \rangle$ is an edge in a directed graph
 - v_0 is **adjacent to** v_1 , and v_1 is **adjacent from** v_0
 - The edge $\langle v_0, v_1 \rangle$ is incident on v_0 and v_1

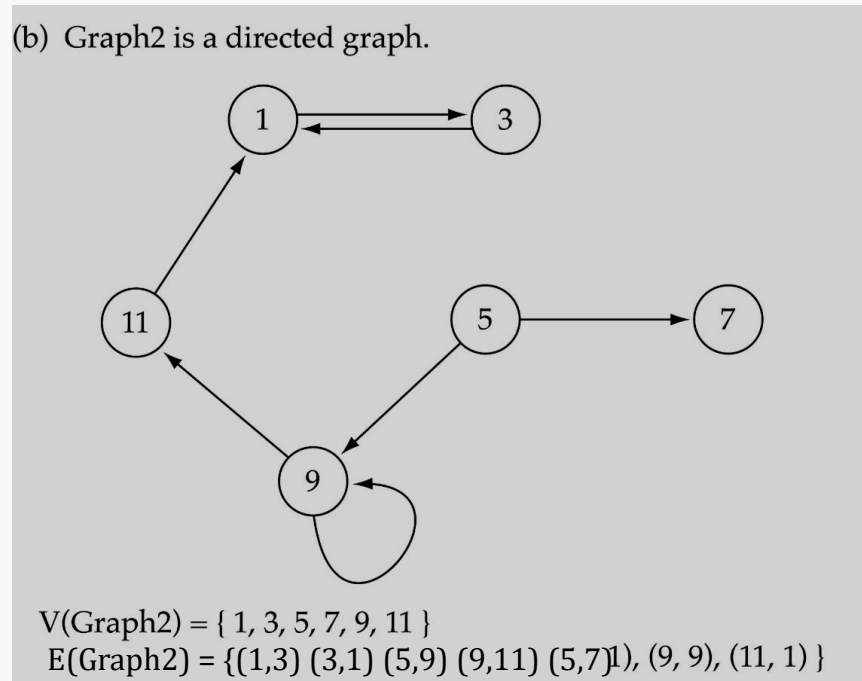
Directed vs. undirected graphs

- When the edges in a graph have no direction, the graph is called *undirected*



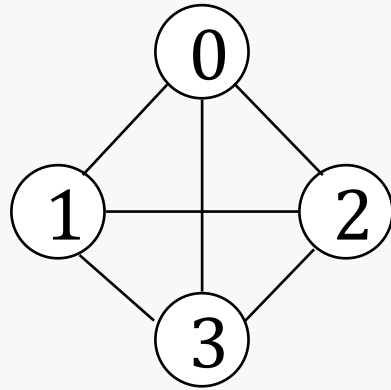
Directed vs. undirected graphs (cont.)

- When the edges in a graph have a direction, the graph is called *directed* (or *digraph*)



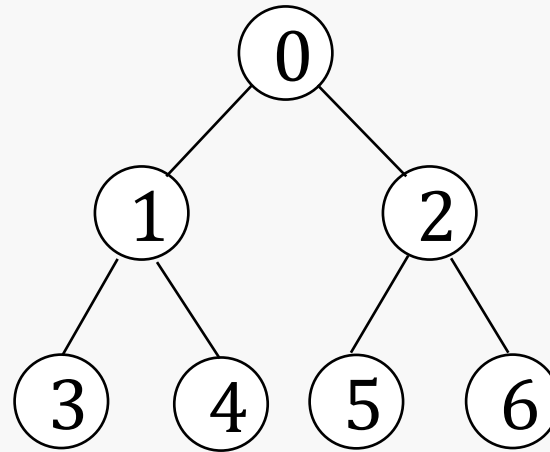
Warning: if the graph is directed, the order of the vertices in each edge is important !!

Examples for Graph



G_1

complete graph



G_2

incomplete graph



G_3

$$V(G_1) = \{0, 1, 2, 3\}$$

$$E(G_1) = \{(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)\}$$

$$V(G_2) = \{0, 1, 2, 3, 4, 5, 6\}$$

$$E(G_2) = \{(0, 1), (0, 2), (1, 3), (1, 4), (2, 5), (2, 6)\}$$

$$V(G_3) = \{0, 1, 2\}$$

$$E(G_3) = \{<0, 1>, <1, 0>, <1, 2>\}$$

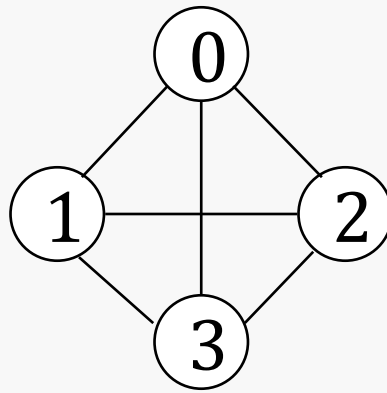
complete undirected graph: $n(n-1)/2$ edges

complete directed graph: $n(n-1)$ edges

Subgraph and Path

- A **subgraph** of G is a graph G' such that $V(G')$ is a subset of $V(G)$ and $E(G')$ is a subset of $E(G)$
- A **path** from vertex v_p to vertex v_q in a graph G , is a sequence of vertices, $v_p, v_{i1}, v_{i2}, \dots, v_{in}, v_q$, such that $(v_p, v_{i1}), (v_{i1}, v_{i2}), \dots, (v_{in}, v_q)$ are edges in an undirected graph
- The **length of a path** is the number of edges on it

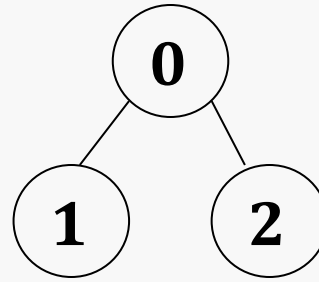
Figure 6.4: subgraphs of G_1 and G_3 (p.261)



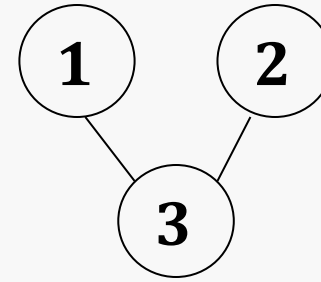
G_1



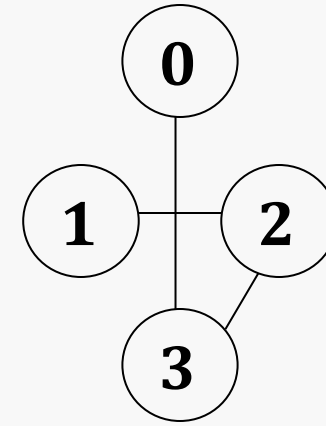
(i)



(ii)



(iii)



(iv)

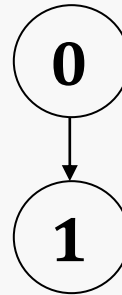
(a) Some of the subgraph of G_1



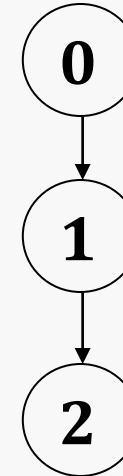
G_3



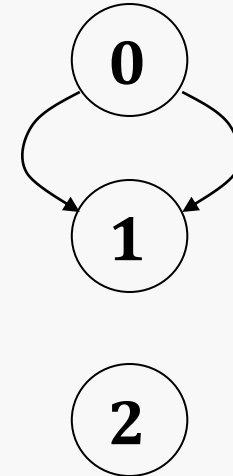
(i)



(ii)



(iii)



(iv)

(b) Some of the subgraph of G_3

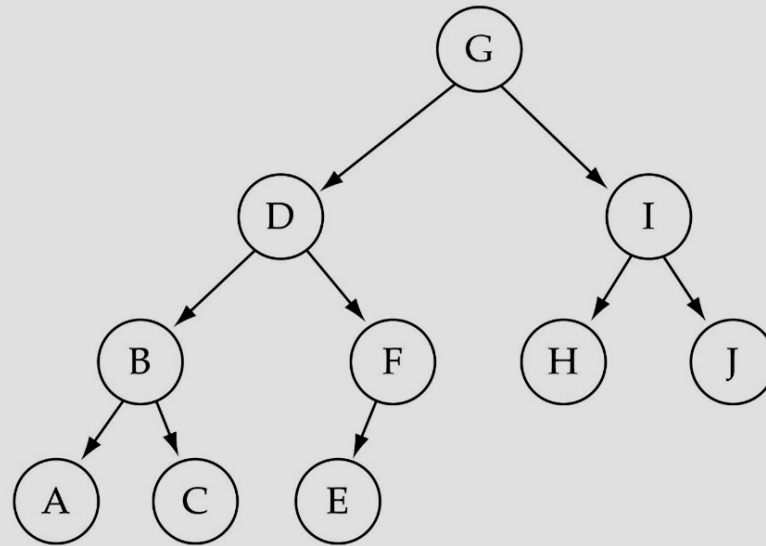
Simple Path and Style

- A **simple path** is a path in which all vertices, except possibly the first and the last, are distinct
- A **cycle** is a simple path in which the first and the last vertices are the same
- In an undirected graph G , two vertices, v_0 and v_1 , are **connected** if there is a path in G from v_0 to v_1
- An undirected graph is **connected** if, for every pair of distinct vertices v_i, v_j , there is a path from v_i to v_j

Trees vs graphs

- Trees are special cases of graphs!!

(c) Graph3 is a directed graph.



$V(\text{Graph3}) = \{ A, B, C, D, E, F, G, H, I, J \}$

$E(\text{Graph3}) = \{ (G, D), (G, I), (D, B), (D, F), (I, H), (I, J), (B, A), (B, C), (F, E) \}$

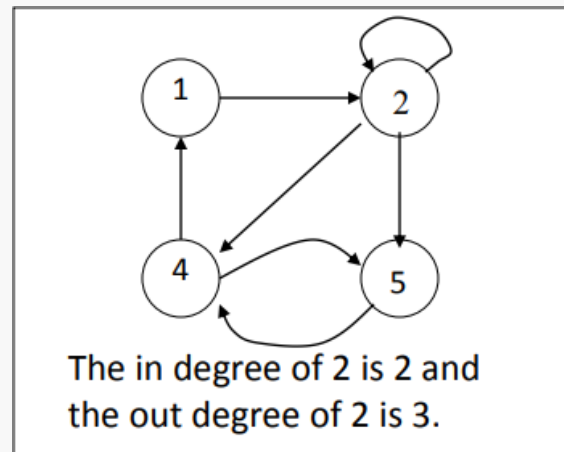
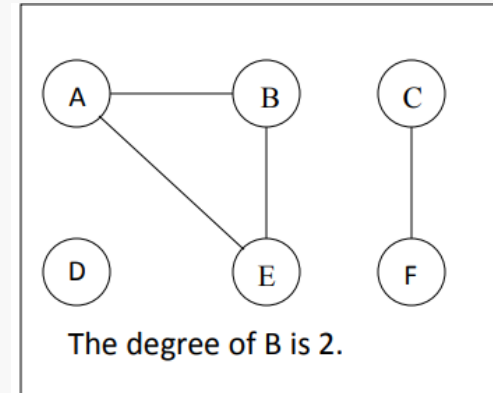
Degree

- The **degree** of a vertex is the number of edges incident to that vertex
- For directed graph,
 - the **in-degree** of a vertex v is the number of edges that have v as the head
 - the **out-degree** of a vertex v is the number of edges that have v as the tail
 - if d_i is the degree of a vertex i in a graph G with n vertices and e edges, the number of edges is

$$e = \left(\sum_{i=0}^{n-1} d_i \right) / 2$$

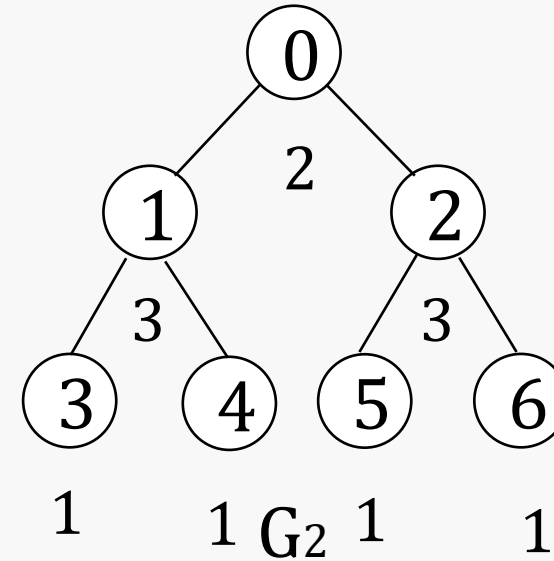
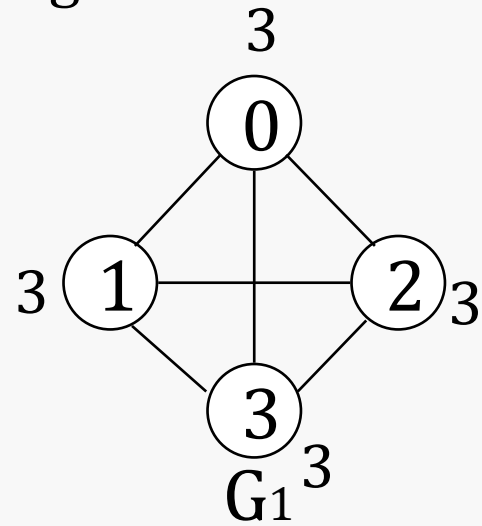
Degree of the Node

- A degree of a node is the number of edges that are connected with that node.
A node with degree 0 is called as isolated node.
- In degree: Number of edges entering a node
 - Out degree: Number of edges leaving a node
- • Degree = Indegree + Outdegree



undirected graph

degree



directed graph

in-degree

out-degree



in:1, out: 1

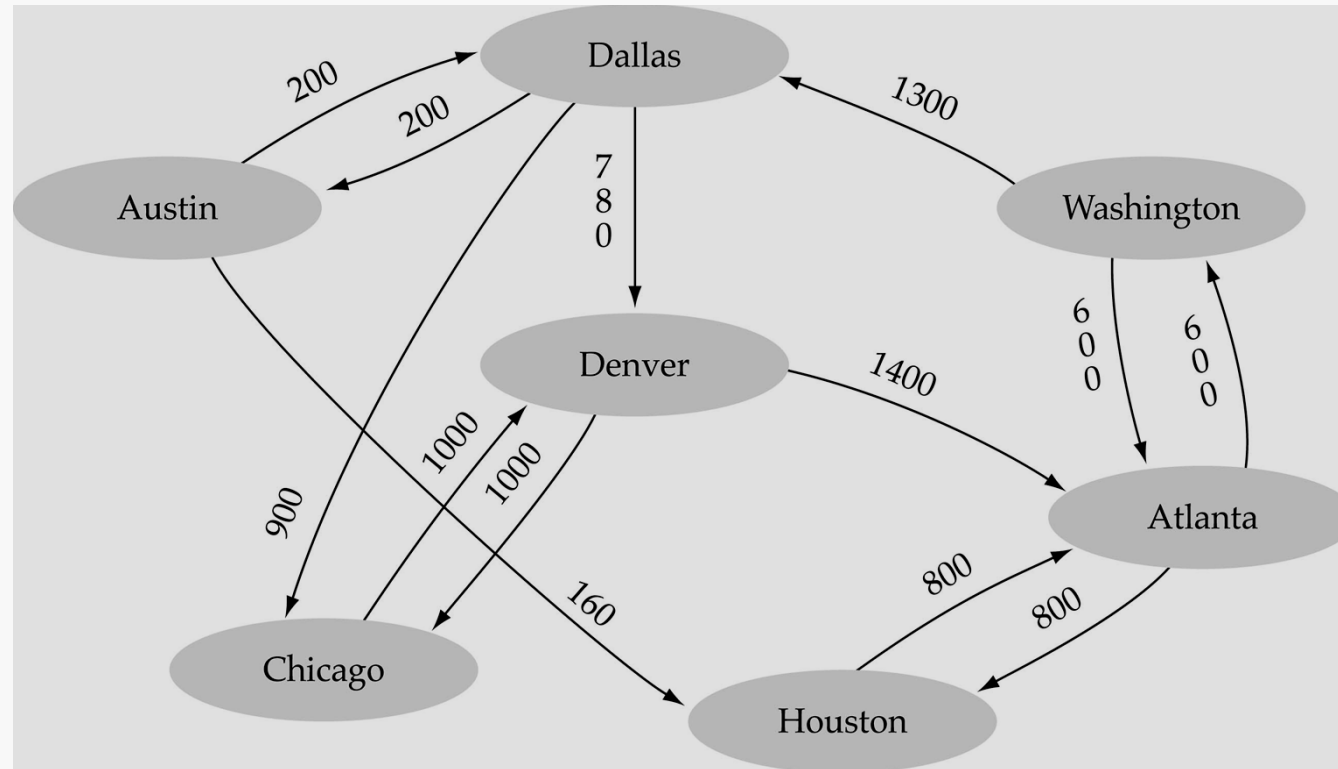
in: 1, out: 2

in: 1, out: 0

G_3

Graph terminology (cont.)

- Weighted graph: a graph in which each edge carries a value



CONT..

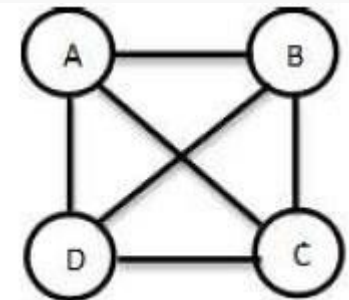
□ **Weighted Graph**

- A graph is said to be weighted graph if every edge in the graph is assigned a weight or value. It can be directed or undirected graph.



□ **Complete Graph**

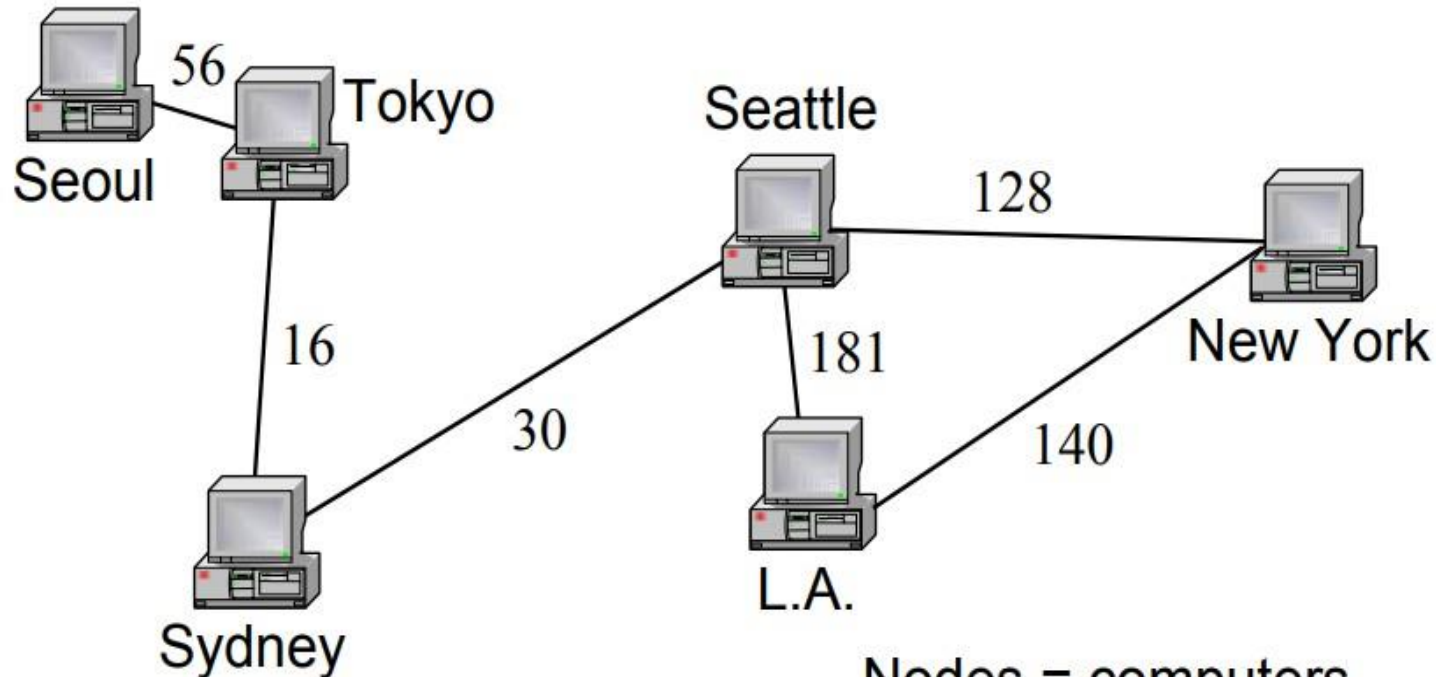
- A complete graph is a graph in which there is a direct edge between every pair of vertices.
- A complete graph with n vertices will have $n(n-1)/2$ edges.
- There is a path from every vertex to every other vertex.
- All complete graphs are connected graphs, but not all connected graphs are complete graphs.
- A complete digraph is a strongly connected graph.



APPLICATIONS

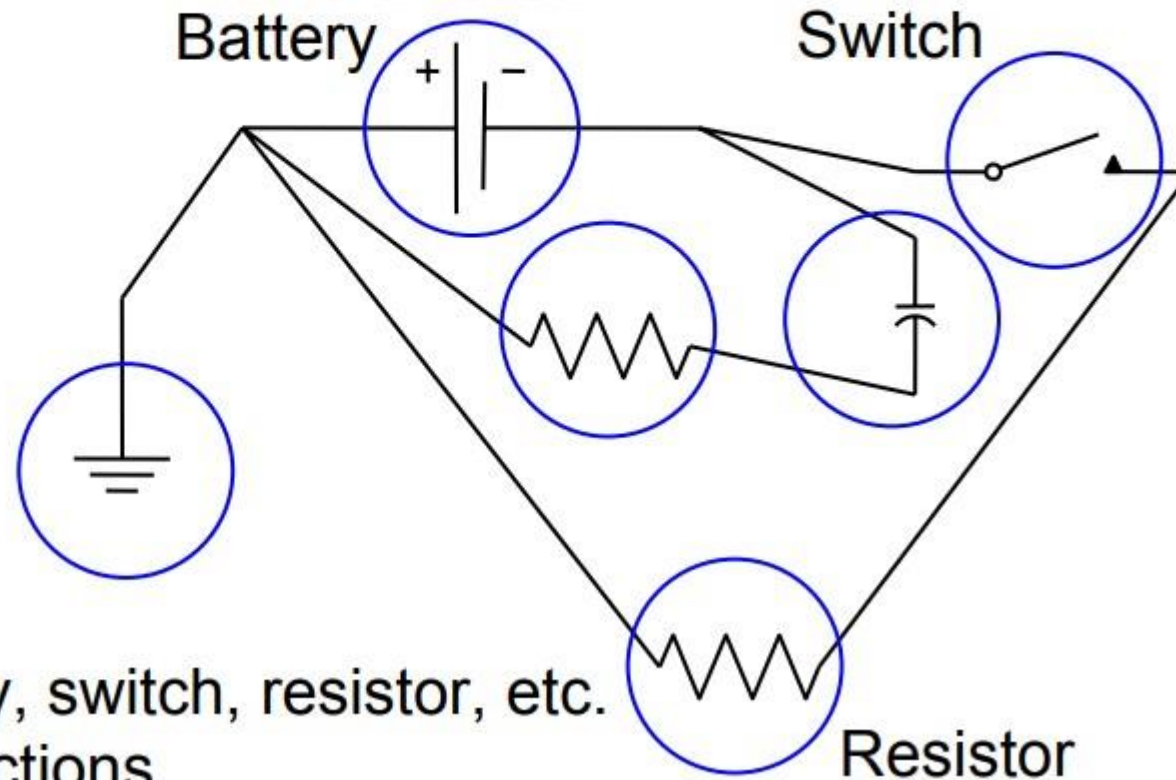
- Graphs are used to solve many real-life problems.
- Graphs are used to represent networks.
- The networks may include paths in a city or telephone network or circuit network.
- Graphs are also used in social networks like linkedIn, Facebook.
- For example, in Facebook, each person is represented with a vertex(or node).
- Each node is a structure and contains information like person id, name, gender, locale etc.

Information Transmission in a Computer Network



Nodes = computers
Edges = transmission rates

Representing Electrical Circuits



Nodes = battery, switch, resistor, etc.
Edges = connections

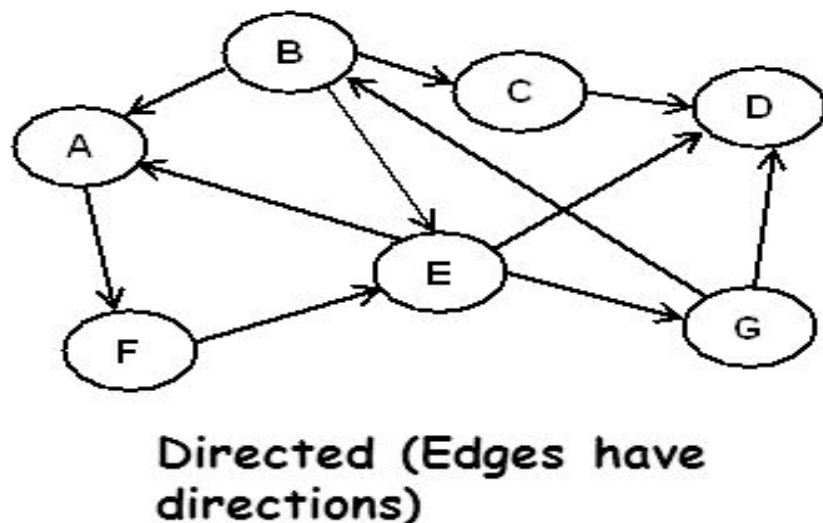
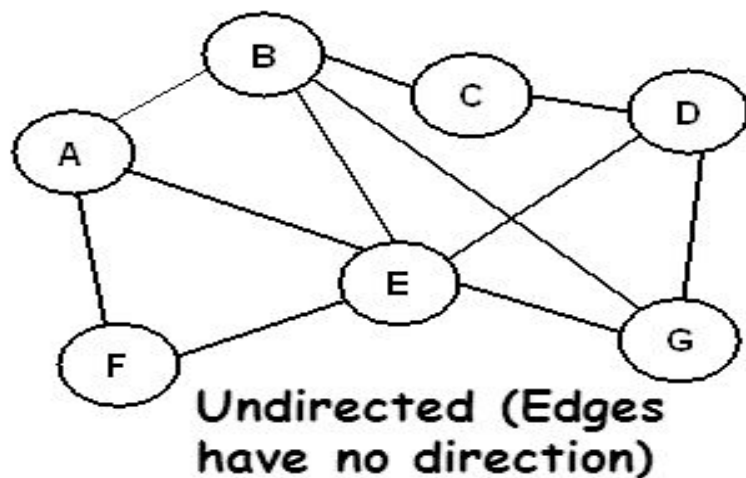
Traffic Flow on Highways



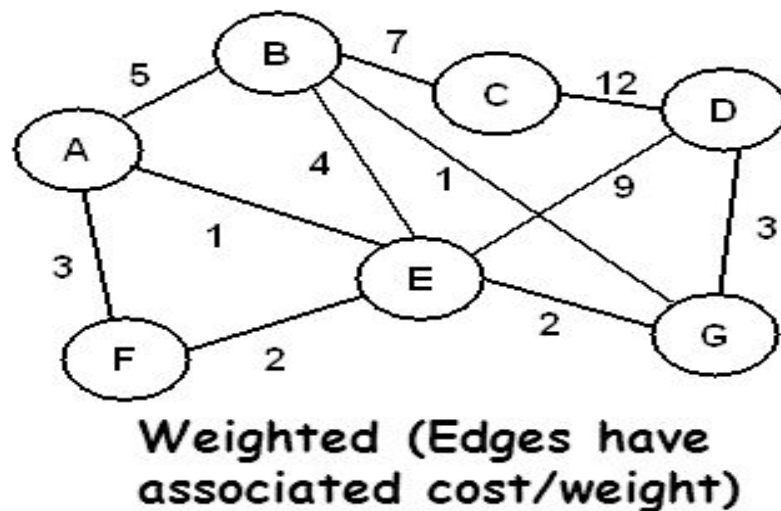
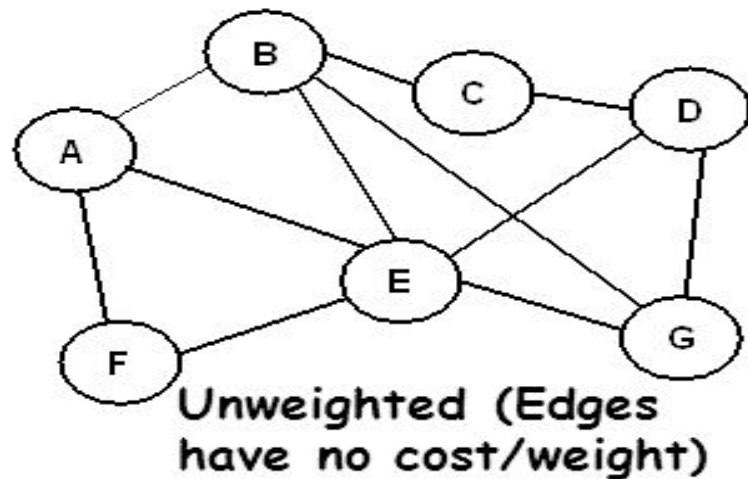
Nodes = cities
Edges = # vehicles on
connecting highway

Types of Graphs

Directed vs. undirected



Weighted vs. unweighted



Adjacency matrix vs. adjacency list representation

■ Adjacency matrix

- *Good for dense graphs -- $|E| \sim O(|V|^2)$*
- *Memory requirements: $O(|V| + |E|) = O(|V|^2)$*
- *Connectivity between two vertices can be tested quickly*

■ Adjacency list

- *Good for sparse graphs -- $|E| \sim O(|V|)$*
- *Memory requirements: $O(|V| + |E|) = O(|V|)$*
- *Vertices adjacent to another vertex can be found quickly*

Graph Representation

- Graphs represented in computer memory
 - *Two common ways*
 - Adjacency matrices
 - Adjacency lists

Adjacency Matrix

- Let $G=(V,E)$ be a graph with n vertices.
- The **adjacency matrix** of G is a two-dimensional n by n array, say `adj_mat`
- If the edge (v_i, v_j) is in $E(G)$, $\text{adj_mat}[i][j]=1$
- If there is no such edge in $E(G)$, $\text{adj_mat}[i][j]=0$
- The adjacency matrix for an undirected graph is symmetric; the adjacency matrix for a digraph need not be symmetric

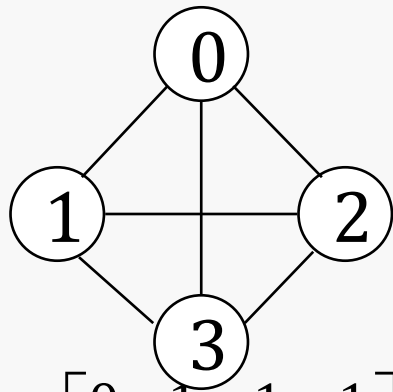
Adjacency Matrices

- Let G be a graph with n vertices where $n > \text{zero}$
- Let $V(G) = \{v_1, v_2, \dots, v_n\}$
 - *Adjacency matrix*

$$A_G(i,j) = \begin{cases} 1 & \text{if } (v_i, v_j) \in E(G) \\ 0 & \text{otherwise} \end{cases}$$

$$A_{G_1} = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad A_{G_2} = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}.$$

Examples for Adjacency Matrix



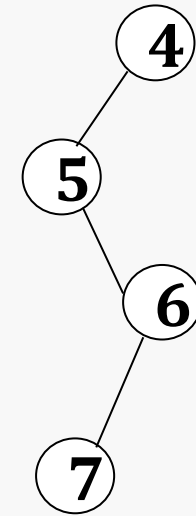
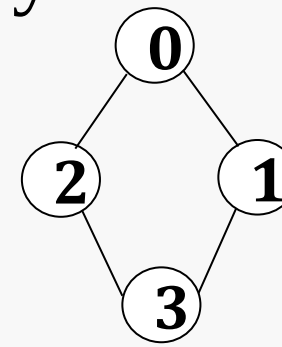
$$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

G_1



$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

G_2



$$\begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

G_4

symmetric

Merits of Adjacency Matrix

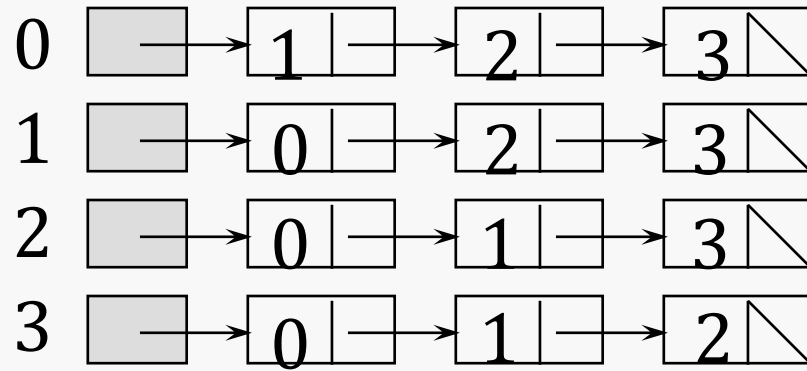
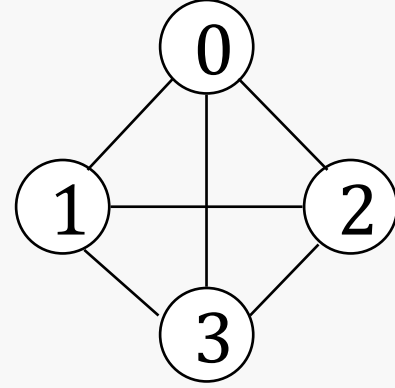
- From the adjacency matrix, to determine the connection of vertices is easy
- The degree of a vertex is $\sum_{j=0}^{n-1} adj_mat[i][j]$
- For a digraph, the row sum is the out_degree, while the column sum is the in_degree

$$ind(v_i) = \sum_{j=0}^{n-1} A[j, i] \quad outd(v_i) = \sum_{j=0}^{n-1} A[i, j]$$

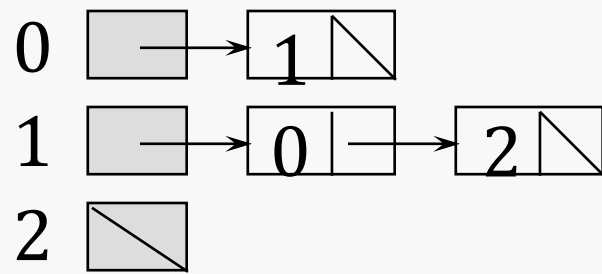
Data Structures for Adjacency Lists

Each row in adjacency matrix is represented as an adjacency list.

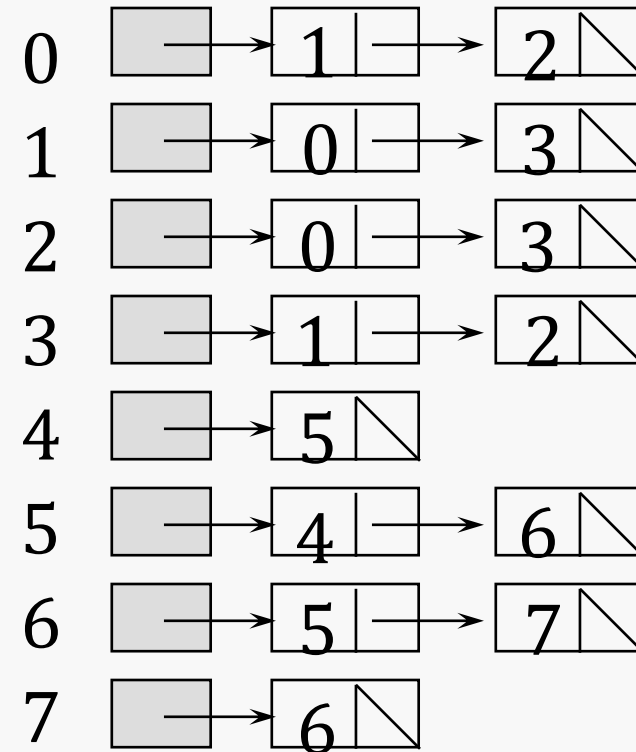
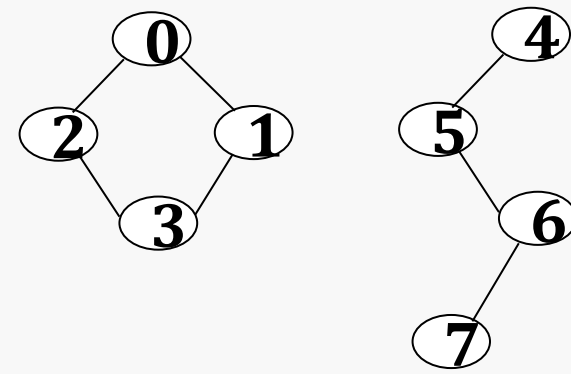
```
#define MAX_VERTICES 50
typedef struct node *node_pointer;
typedef struct node {
    int vertex;
    struct node *link;
};
node_pointer graph[MAX_VERTICES];
int n=0; /* vertices currently in use */
```



G_1



G_3



G_4

An undirected graph with n vertices and e edges $\implies n$ head nodes and $2e$ list nodes

Adjacency Lists

- Given:
 - *Graph G with n vertices, where $n > \text{zero}$*
 - *$V(G) = \{v_1, v_2, \dots, v_n\}$*
- For each vertex v : linked list exists
 - *Linked list node contains vertex u : $(v, u) \in E(G)$*
- Use array A , of size n , such that $A[i]$
 - *Reference variable pointing to first linked list node containing vertices to which v_i adjacent*
- Each node has two components: vertex, link
 - *Component vertex*
 - Contains index of vertex adjacent to vertex i

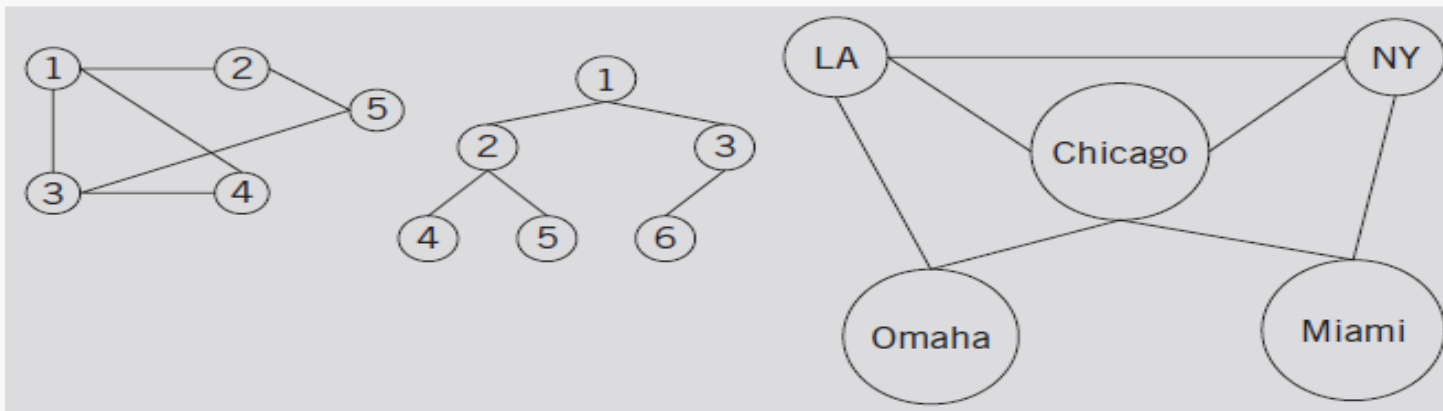


FIGURE 12-3 Various undirected graphs

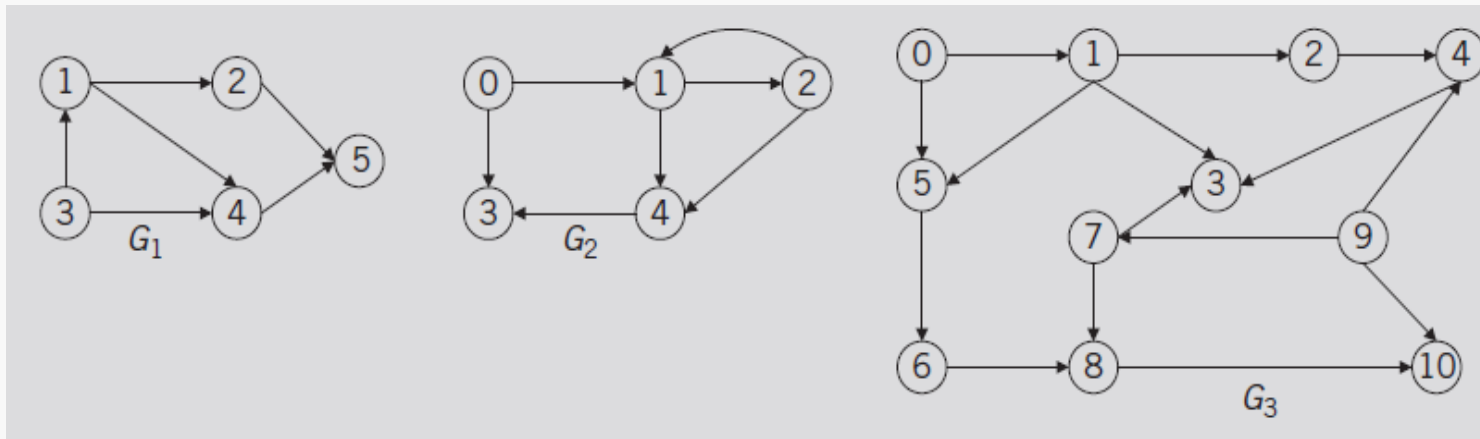


FIGURE 12-4 Various directed graphs

$V(G_1) = \{1, 2, 3, 4, 5\}$

$V(G_2) = \{0, 1, 2, 3, 4\}$

$V(G_3) = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$

$E(G_1) = \{(1, 2), (1, 4), (2, 5), (3, 1), (3, 4), (4, 5)\}$

$E(G_2) = \{(0, 1), (0, 3), (1, 2), (1, 4), (2, 1), (2, 4), (4, 3)\}$

$E(G_3) = \{(0, 1), (0, 5), (1, 2), (1, 3), (1, 5), (2, 4), (4, 3), (5, 6), (6, 8), (7, 3), (7, 8), (8, 10), (9, 4), (9, 7), (9, 10)\}$

Adjacency Lists (cont'd.)

- Example 12-4

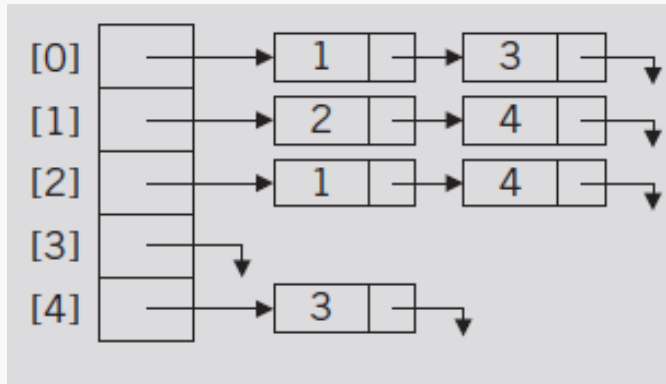


FIGURE 12-5 Adjacency list of graph G2 of Figure 12-4

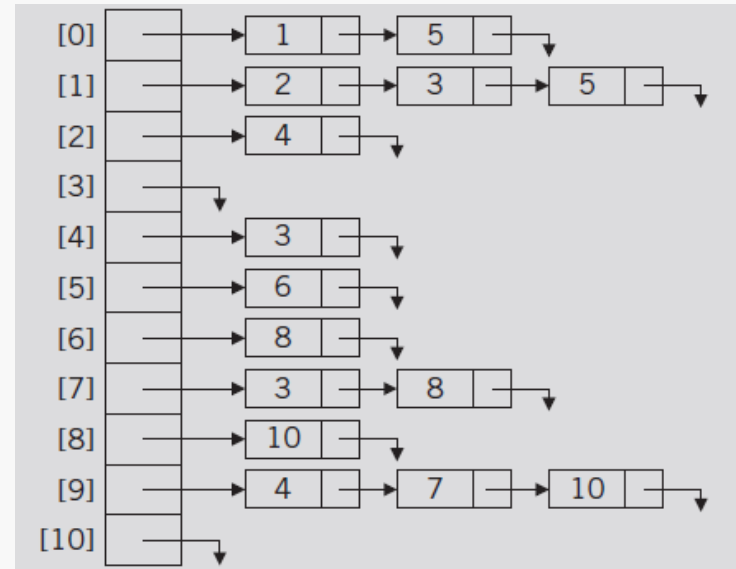
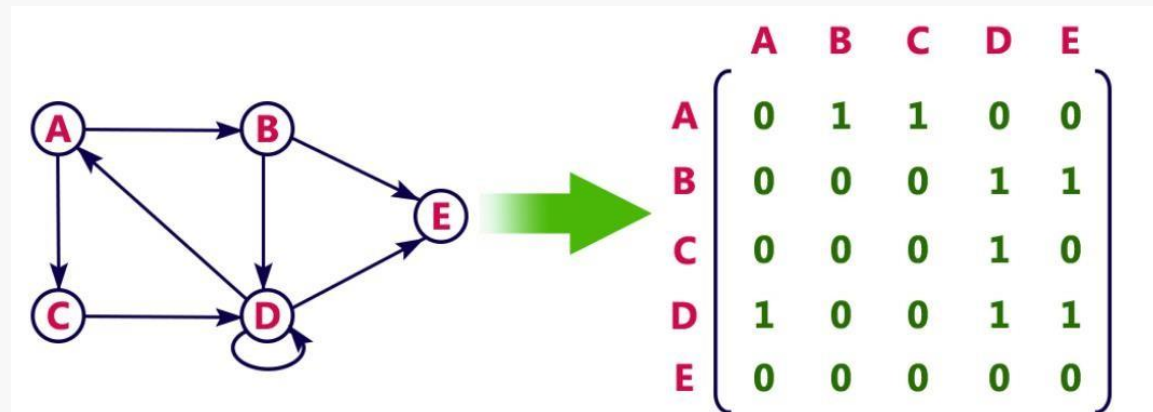
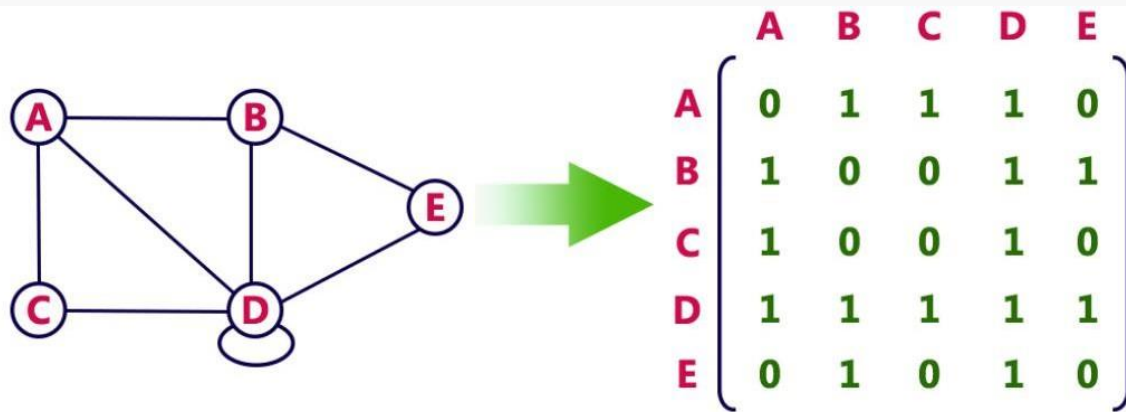


FIGURE 12-6 Adjacency list of graph G3 of Figure 12-4

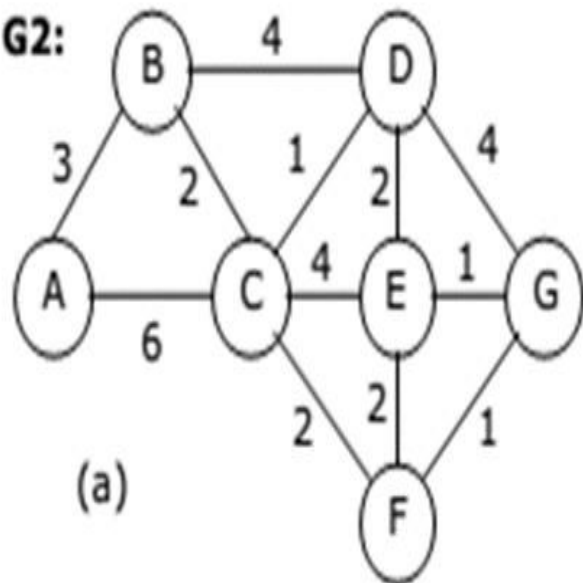
Some more Examples.....

ADJACENCY MATRIX

- The adjacency matrix A for a graph $G = (V, E)$ with n vertices, is an $n \times n$ matrix of bits ,
- such that $A_{ij} = 1$, if there is an edge from v_i to v_j and
- $A_{ij} = 0$, if there is no such edge



G2:

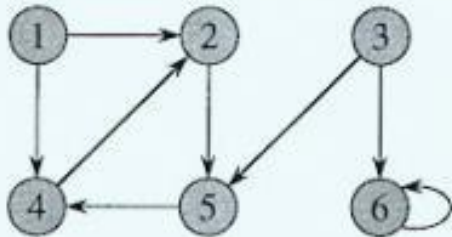


(b)

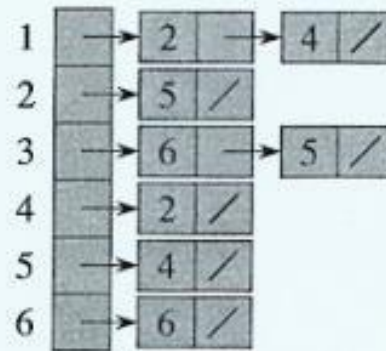
	A	B	C	D	E	F	G
A	0	3	6	∞	∞	∞	∞
B	3	0	2	4	∞	∞	∞
C	6	2	0	1	4	2	∞
D	∞	4	1	0	2	∞	4
E	∞	∞	4	2	0	2	1
F	∞	∞	2	∞	2	0	1
G	∞	∞	∞	4	1	1	0

ADJACENCY LIST

- A graph containing m vertices and n edges can be represented using a linked list, referred to as *adjacency list*.
- The number of vertices in a graph forms a singly linked list.
- Each vertex have a separate linked list, with nodes equal to the number of edges connected from the corresponding vertex..
- Each nodes has at least 2 fields: VERTEX and LINK.
- The VERTEX fields contain the indices of the vertices adjacent to vertex i .



(a)

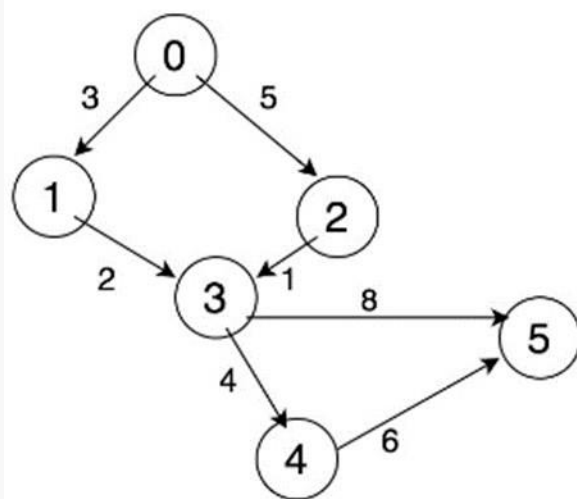


(b)

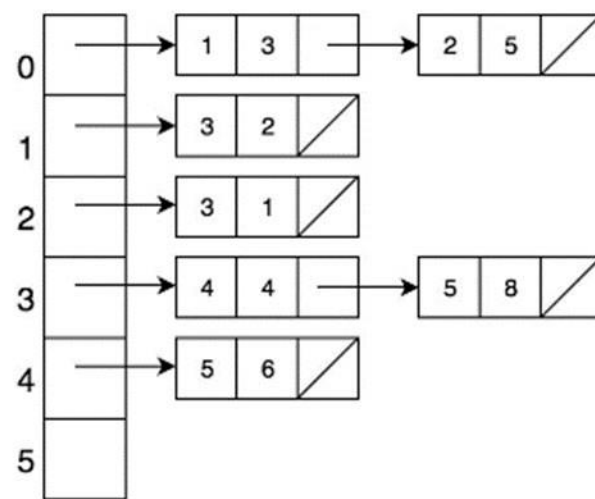
	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

(c)

Directed Graph



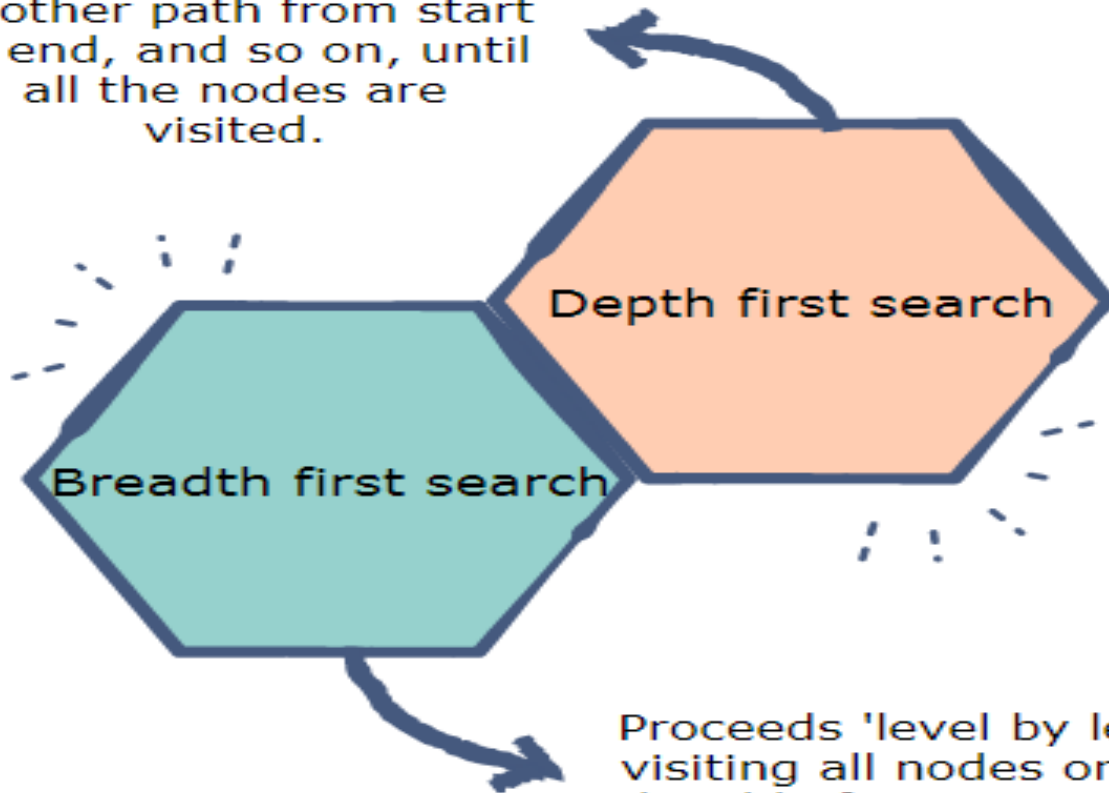
Adjacency List Representation



Graph Traversals

- A graph traversal is a systematic way of visiting the nodes in a specific order.
- There are 2 types of graph traversals namely,
 - □ Breadth First Search(BFS)
 - □ Depth First Search(DFS)

Follows a path from the starting node to an ending node, then another path from start to end, and so on, until all the nodes are visited.



Proceeds 'level by level', visiting all nodes on one level before moving on to the next.

Depth first search

- Visit the first node initially, and then find the unvisited node which is adjacent to the first node, is visited and a DFS is initiated from the adjacent node (considering it as the first node).
- If all the adjacent nodes have been visited, backtrack to the last node visited, and find another adjacent node and again initiate the DFS from adjacent node.
- This traversal continues until all nodes have been visited once.

Steps to Implement DFS

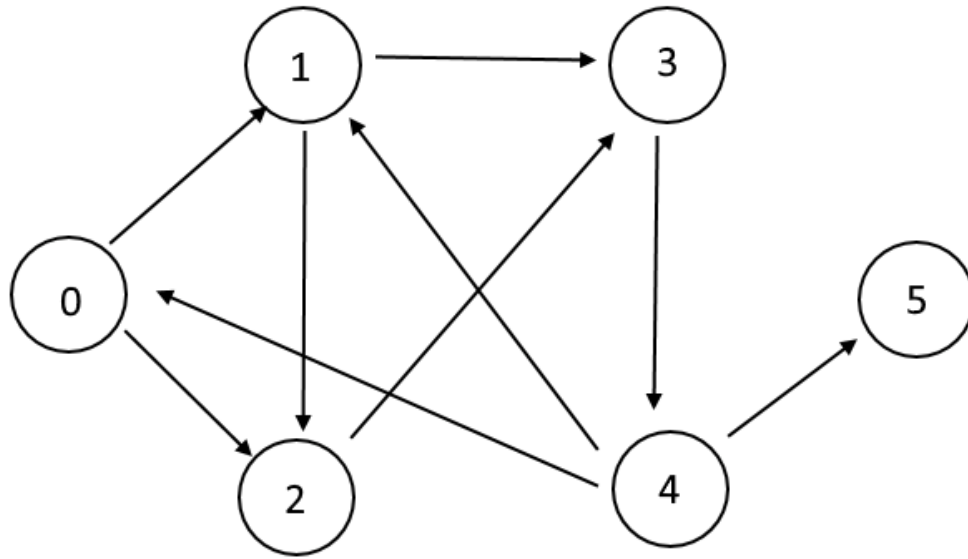
- 1. Select the start vertex
- 2. Visit the vertex (place 1)
- 3. Find the adjacent vertices of visited node
 - **Rule 1** – Visit any one of the adjacent unvisited vertex. Mark it as visited. Display it. Push it in a stack.
 - **Rule 2** – If no adjacent vertex is found, pop up a vertex from the stack. (It will pop up all the vertices from the stack, which do not have adjacent vertices.)
 - **Rule 3** – Repeat Rule 1 and Rule 2 until the stack is empty.

Pseudo code

```
DFS-iterative (G, s):           //Where G is graph and s is source vertex
    let S be stack
    S.push( s )                 //Inserting s in stack
    mark s as visited.
    while ( S is not empty):
        //Pop a vertex from stack to visit next
        v = S.top( )
        S.pop( )
        //Push all the neighbours of v in stack that are not visited
        for all neighbours w of v in Graph G:
            if w is not visited :
                S.push( w )
                mark w as visited
```

Approach:

- Use Stack.
- First add the Starting Node to the Stack.
- Pop out an element from Stack and add all of its connected nodes to stack.
- Repeat the above two steps until the Stack is empty.
- Below is a walk through of the graph above.



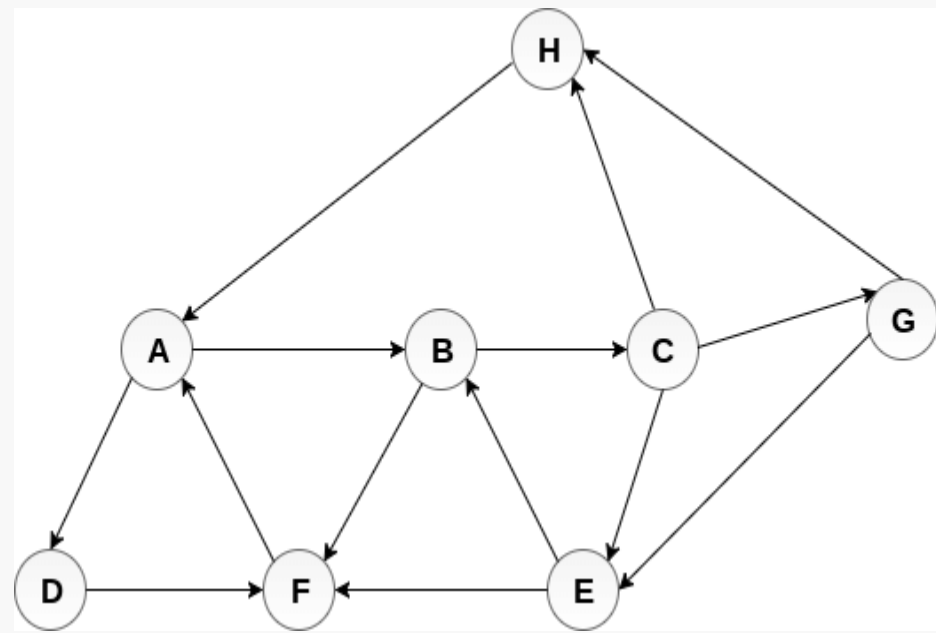
Depth First Traversal - 0 1 3 4 5 2

Stack

push 0	0
pop 0 and push 2, 1	2, 1
pop 1 and push 3	3, 2
pop 3 and push 4	4, 2
pop 4 and push 5	5, 2
pop 5	2
pop 2	

DFS

0
0 1
0 1 3
0 1 3 4
0 1 3 4 5
0 1 3 4 5 2



Adjacency Lists

A : B, D

B : C, F

C : E, G, H

G : E, H

E : B, F

F : A

D : F

H : A

Push H onto the stack

STACK : H

POP the top element of the stack i.e. H, print it and push all the neighbours of H onto the stack that are in ready state.

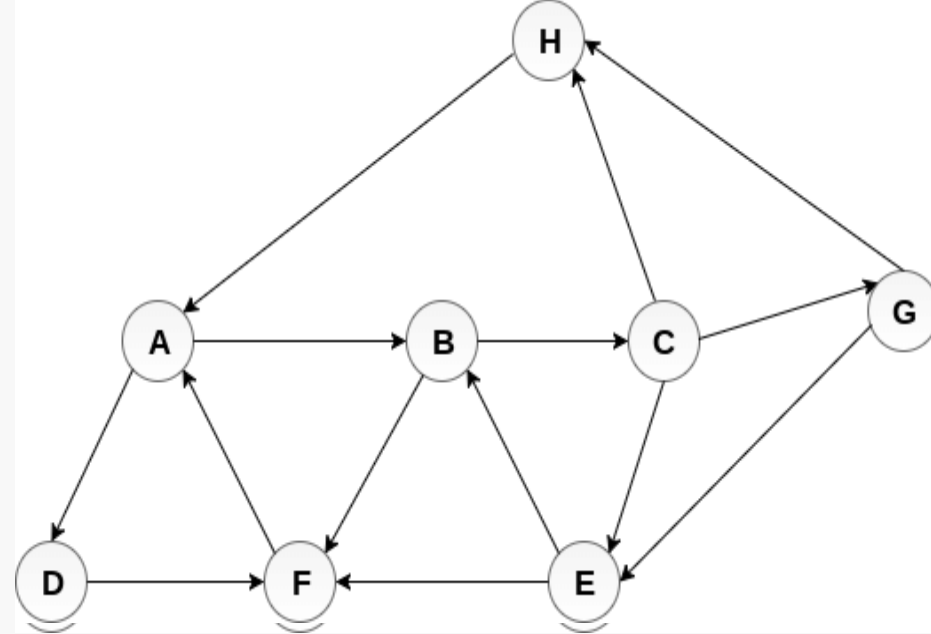
Print H

STACK : A

Pop the top element of the stack i.e. A, print it and push all the neighbours of A onto the stack that are in ready state.

Print A

Stack : B, D



Adjacency Lists

A : B, D

B : C, F

C : E, G, H

G : E, H

E : B, F

F : A

D : F

H : A

Pop the top element of the stack i.e. D, print it and push all the neighbours of D onto the stack that are in ready state.

Print D

Stack : B, F

Pop the top element of the stack i.e. F, print it and push all the neighbours of F onto the stack that are in ready state.

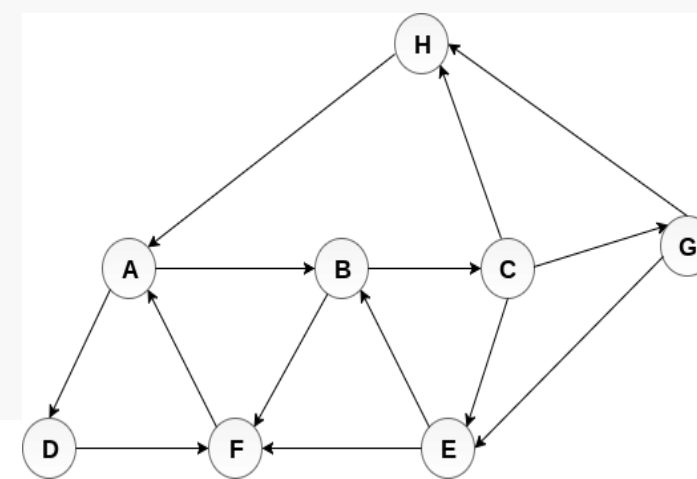
Print F

Stack : B

Pop the top of the stack i.e. B and push all the neighbours

Print B

Stack : C



Adjacency Lists

A : B, D
B : C, F
C : E, G, H
G : E, H
E : B, F
F : A
D : F
H : A

Pop the top of the stack i.e. C and push all the neighbours.

Print C
Stack : E, G

Pop the top of the stack i.e. G and push all its neighbours.

Print G
Stack : E

Pop the top of the stack i.e. E and push all its neighbours.

Print E
Stack :

Hence, the stack now becomes empty and all the nodes of the graph have been traversed.

The printing sequence of the graph will be :

H → A → D → F → B → C → G → E

Applications of DFS

- □ To check whether the undirected graph is connected or not
- □ To check if the connected undirected graph is bi-connected or not
- □ To check whether the directed graph is a-cyclic or not

BFS (Breadth First Search)

- Breadth First Search (BFS) of a graph G starts from an unvisited vertex u .
- Then all unvisited vertices v_i adjacent to u are visited and then all unvisited vertices w_j adjacent to v_i are visited and so on.
- The traversal terminates when there are no more nodes to visit.
- BFS uses a queue data structure to keep track of the order of the nodes whose adjacent nodes are to be visited.

Steps to Implement BFS

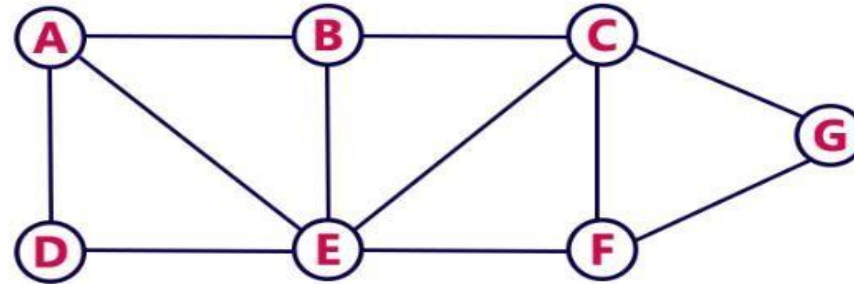
- ❑ 1. Select the start vertex and mark it as visited (i.e) place the value 1
- ❑ 2. Enqueue the START vertex.
- ❑ 3. Dequeue the vertex.
- ❑ 4. Find all adjacent unvisited vertices of the dequeued vertex.
- ❑ 5. Mark all unvisited adjacent vertices as visited.
- ❑ 6. Enqueue all adjacent vertices.
- ❑ 7. Repeat from step 3 to step 6 until the queue becomes empty

Pseudo Code For BFS

```
void BFS ( Vertex S )
{
    Vertex V,W;
    Queue Q;
    visited[S] = 1;
    enqueue(S,Q);
    while(!(Isempty(Q))
    {
        V=dequeue(Q);
        print(V);
        for each W adjacent to V
        if ( visited[w] == 0 )
        {
            visited[W] = 1;
            enqueue(W,Q);
        }
    }
}
```

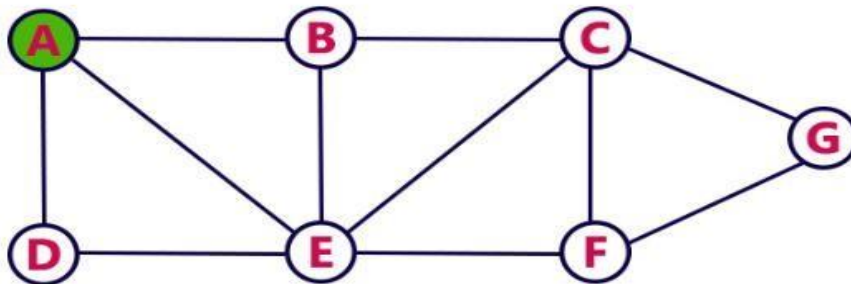
EXAMPLE 2

Consider the following example graph to perform BFS traversal



Step 1:

- Select the vertex **A** as starting point (visit **A**).
- Insert **A** into the Queue.



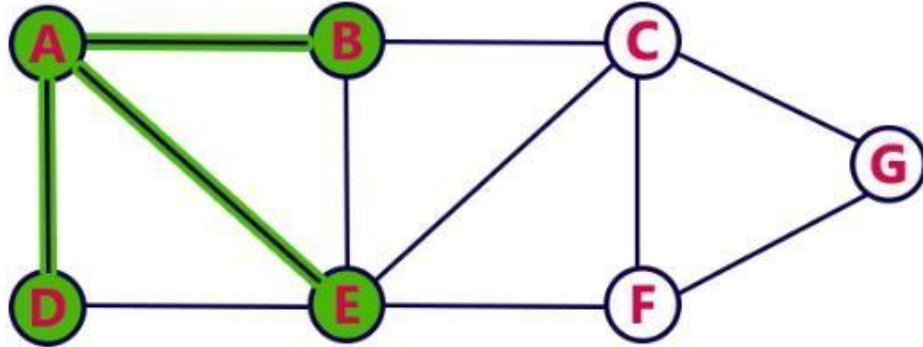
Queue



Front, rear
↑

Step 2:

- Visit all adjacent vertices of **A** which are not visited (**D**, **E**, **B**).
- Insert newly visited vertices into the Queue and delete A from the Queue..



Queue

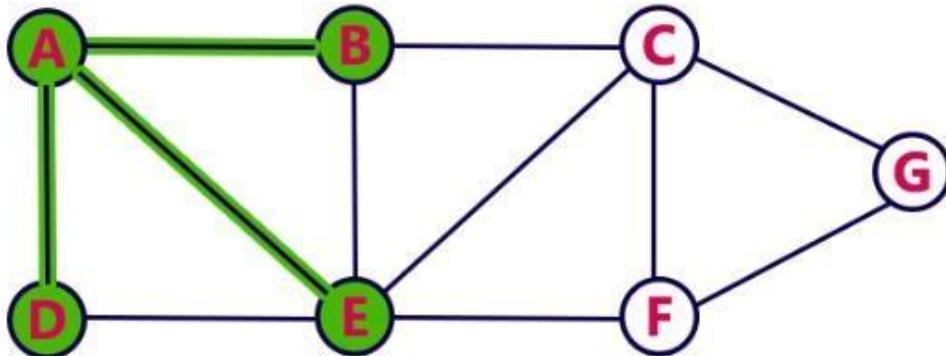


FRONT REAR



Step 3:

- Visit all adjacent vertices of **D** which are not visited (there is no vertex).
- Delete D from the Queue.



Queue

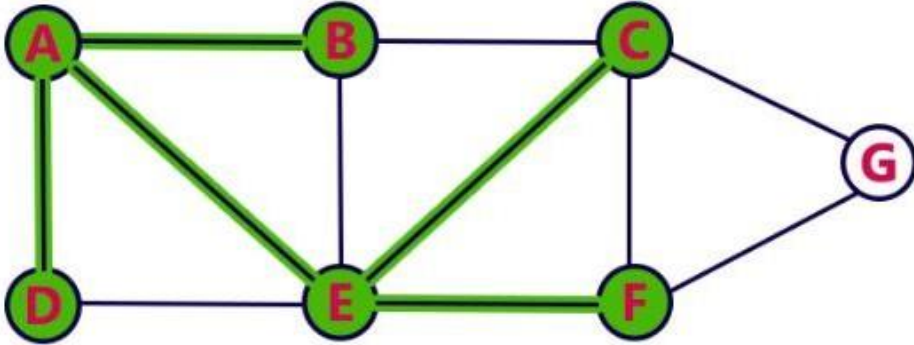


FRONT REAR



Step 4:

- Visit all adjacent vertices of **E** which are not visited (**C, F**).
- Insert newly visited vertices into the Queue and delete E from the Queue.



Queue



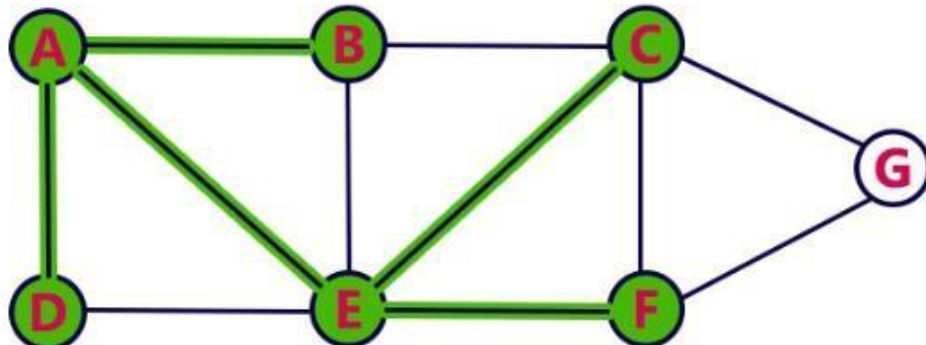
FRONT

REAR



Step 5:

- Visit all adjacent vertices of **B** which are not visited (**there is no vertex**).
- Delete **B** from the Queue.



Queue



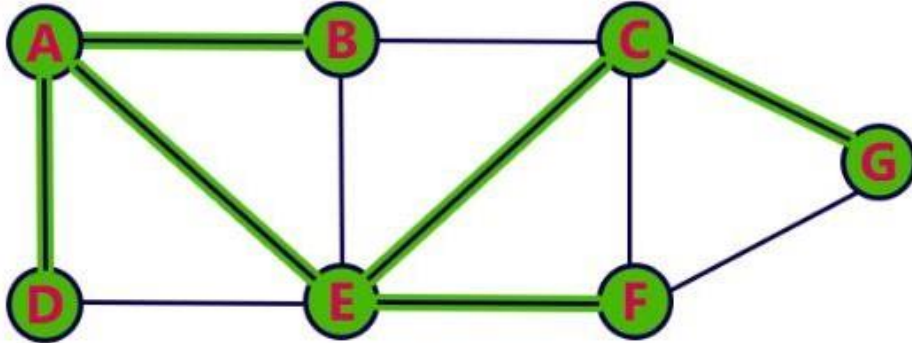
FRONT

REAR



Step 6:

- Visit all adjacent vertices of **C** which are not visited (**G**).
- Insert newly visited vertex into the Queue and delete **C** from the Queue.



Queue

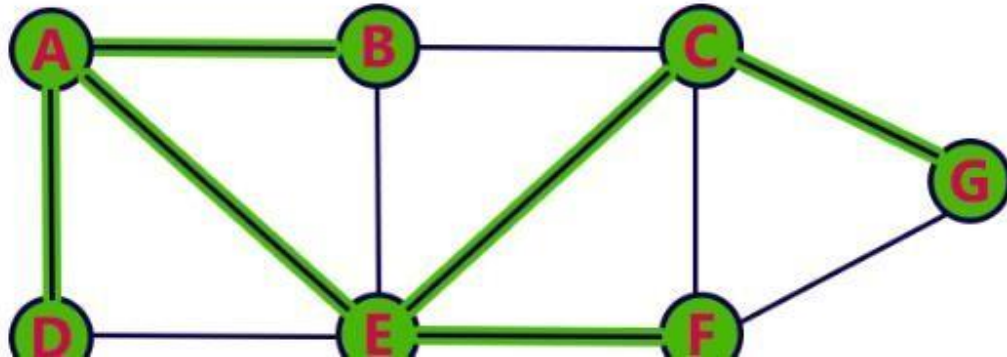


FRONT REAR



Step 7:

- Visit all adjacent vertices of **F** which are not visited (**there is no vertex**).
- Delete **F** from the Queue.

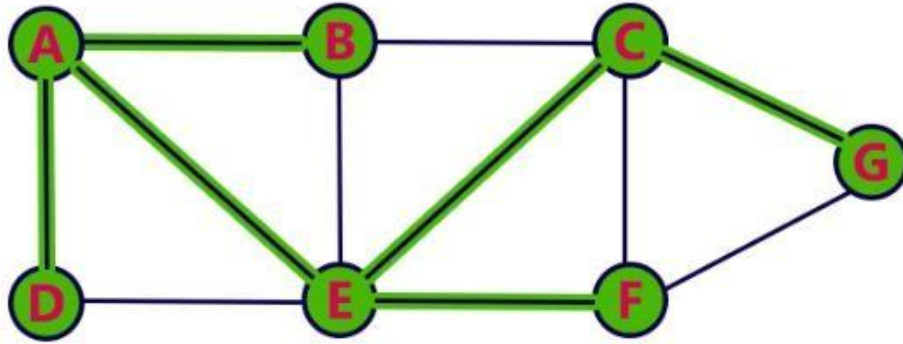


Queue



Step 8:

- Visit all adjacent vertices of **G** which are not visited (**there is no vertex**).
- Delete **G** from the Queue.

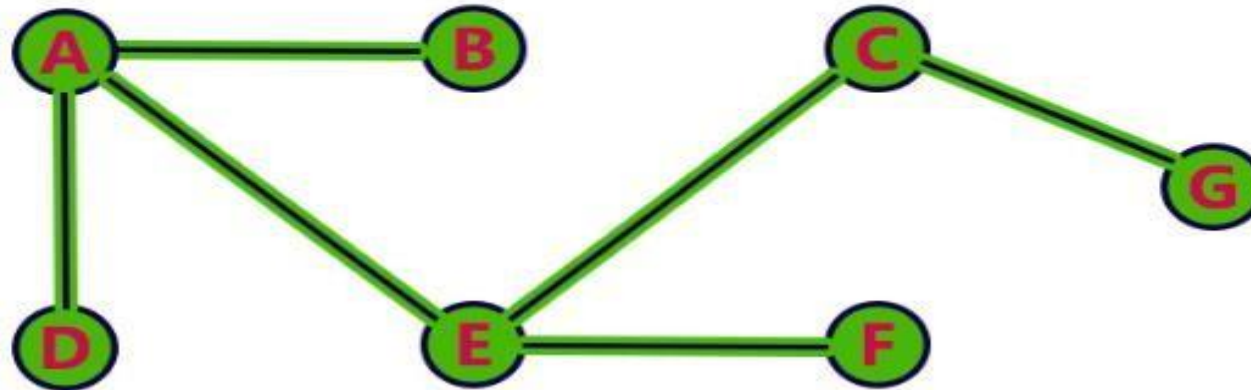


Queue



Front=1, rear=-1

- Queue became Empty. So, stop the BFS process.
- Final result of BFS is a Spanning Tree as shown below...



A-D-B-E-C-F-G

Applications of BFS

- 1. To find the shortest path from a vertex s to a vertex v in an unweighted graph
- 2. To find the length of such a path
- 3. To find out if a graph contains cycles
- 4. To construct a BFS tree/forest from a graph

Comparison between DFS and BFS

DFS	BFS
DFS visit nodes of graph depth wise. It visits nodes until reach a leaf or a node which doesn't have non-visited nodes.	BFS visit nodes level by level in Graph.
Usually implemented using a stack data structure.	Usually implemented using a queue data structure.
Generally requires less memory than BFS.	Generally requires more memory than DFS.
Not Optimal for finding the shortest distance.	optimal for finding the shortest distance.
DFS is better when target is far from source.	BFS is better when target is closer to source.
VELAMMAL ENGINEERING COLLEGE, Dept. of CSE	5/29/2020 52

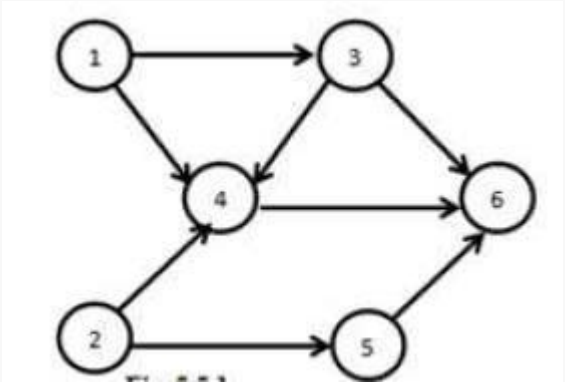
TOPOLOGICAL SORTING

- It is a linear ordering of vertices in a directed a-cyclic graph, such that if there is a path from V_i to V_j , then V_j appears after V_i in the linear ordering.
- Topological sort is not possible if the graph has a cycle.
- **Procedure**
 - 1. Find the indegree for every vertex
 - 2. Place the vertices whose indegree is zero on the empty queue
 - 3. Dequeue one vertex at a time from the queue and decrement the indegree of all its adjacent vertices
 - 4. Enqueue a vertex to the queue if its indegree falls to zero
 - 5. Repeat from step step 3 unitl the queue becomes empty
- The topological ordering is the order in which the vertices are dequeued.

Pseudo Code For Topological Sort

```
void Topsort(Graph G)
{
    Queue Q;
    Vertex V,W;
    int counter=0;
    Q = Create_Queue(NumVertex);
    MakeEmpty(Q);
    for each vertex V
        if ( Indegree[V] == 0 )
            enqueue(V,Q);
    while( !IsEmpty(Q) )
    {
        V = Dequeue(Q);
        Topnum[V] = ++counter;
        for each W adjacent to V
            if ( --Indegree[W] == 0)
                enqueue(W,Q);
    }
    if ( counter != NumVertex )
        error("Graph has a cycle!...");
    DisposeQueue(Q);
}
```

EXAMPLE



- Step 1
- Find the Indegree of vertices 1,2,3,4,5,6.
- Indegree of a vertex is the number of edges entering into the vertex. Indegree of vertex 1 is 0, vertex 2 is 0, vertex 3 is 1, vertex 4 is 3, vertex 5 is 1, vertex 6 is 3.

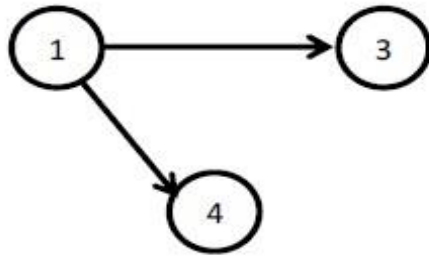
Vertices	Indegree
1	0
2	0
3	1
4	3
5	1
6	3
enqueue()	
dequeue()	

- Step 2
- enqueue() the vertices with Indegree 0. Therefore enqueue() vertices 1 and 2.

Vertices	Indegree
1	0
2	0
3	1
4	3
5	1
6	3
enqueue()	1,2
dequeue()	

Step 3

dequeue() vertex 1 and find all the adjacent vertices. The adjacent vertices are 3 and 4. Therefore decrement the Indegree of the adjacent vertices 3 and 4 by 1.

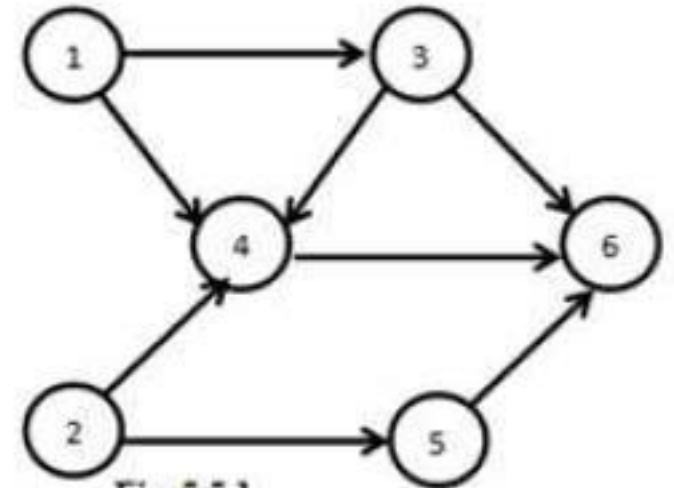


Vertices	Indegree	
1	0	
2	0	
3	1	0
4	3	2
5	1	
6	3	
enqueue()	1,2	
dequeue()	1	

Step 4

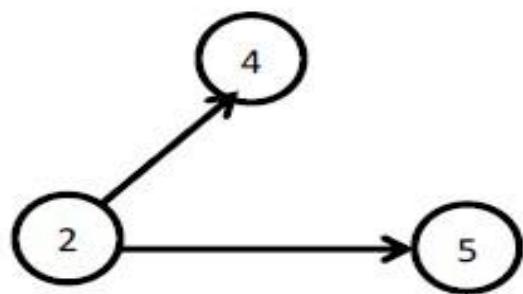
enqueue() the vertex 3 since its Indegree is 0. dequeue() the vertex 2.

Vertices	Indegree	
1	0	
2	0	
3	1	0
4	3	2
5	1	
6	3	
enqueue()	1,2	3
dequeue()	1	2



Step 5

Find the adjacent vertices of vertex 2. Decrement the Indegree of the adjacent vertices 4 and 5 by 1.

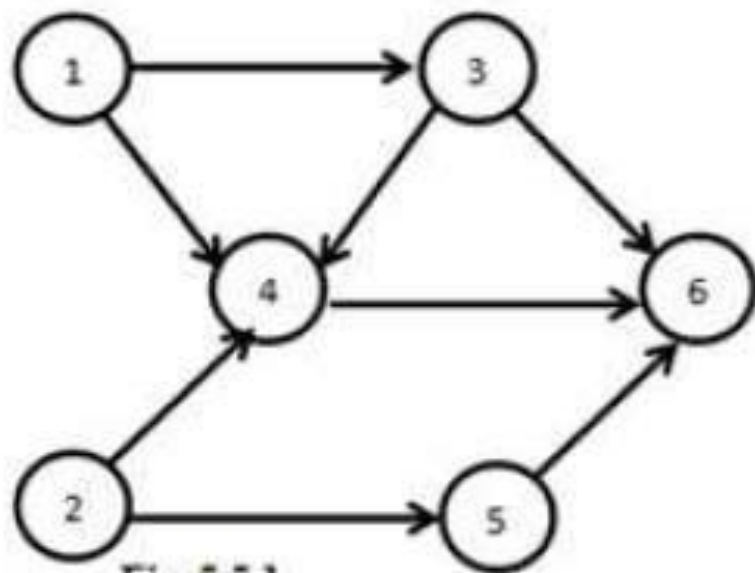


Vertices	Indegree		
1	0		
2	0		
3	1	0	1
4	3	2	0
5	1		
6	3		
enqueue()	1,2	3	
dequeue()	1	2	

Step 6

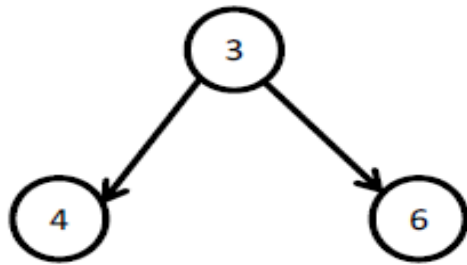
enqueue() the vertex 5 since its indegree is 0. dequeue() the vertex 3.

Vertices	Indegree		
1	0		
2	0		
3	1	0	1
4	3	2	0
5	1		
6	3		
enqueue()	1,2	3	5
dequeue()	1	2	3

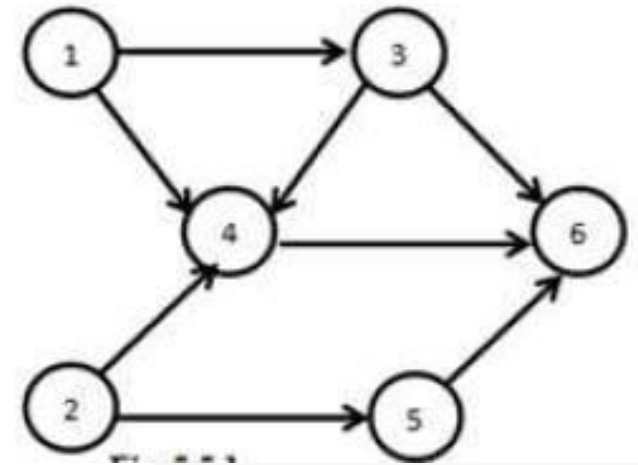


Step 7

Find the adjacent vertices of the dequeued vertex 3. Decrement the Indegree of the vertices 4 and 6 by 1.



Vertices	Indegree			
1	0			
2	0			
3	1	0		
4	3	2	1	0
5	1		0	
6	3			2
enqueue()	1,2	3	5	
dequeue()	1	2	3	



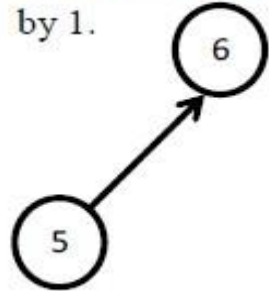
Step 8

enqueue() the vertex 4, because its Indegree is 0. dequeue() the vertex 5.

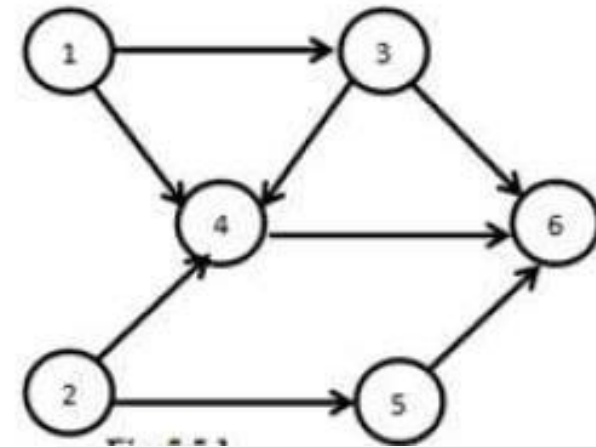
Vertices	Indegree			
1	0			
2	0			
3	1	0		
4	3	2	1	0
5	1		0	
6	3			2
enqueue()	1,2	3	5	4
dequeue()	1	2	3	5

Step 9

Find the adjacent vertices of the dequeued vertex 5. Decrease the Indegree of the vertex 6 by 1.



Vertices	Indegree				
1	0				
2	0				
3	1	0			
4	3	2	1	0	
5	1		0		
6	3			2	1
enqueue()	1,2	3	5	4	
dequeue()	1	2	3	5	



Step 10

dequeue() the vertex 4 whose Indegree is 0 and find the adjacent vertices of the vertex 4.



Vertices	Indegree				
1	0				
2	0				
3	1	0			
4	3	2	1	0	
5	1		0		
6	3			2	1
enqueue()	1,2	3	5	4	
dequeue()	1	2	3	5	4

Step 11

Decrement the Indegree of the vertex 6 by 1.

Vertices	Indegree					
1	0					
2	0					
3	1	0				
4	3	2	1	0		
5	1		0			
6	3			2	1	0
enqueue()	1,2	3	5	4		
dequeue()	1	2	3	5	4	

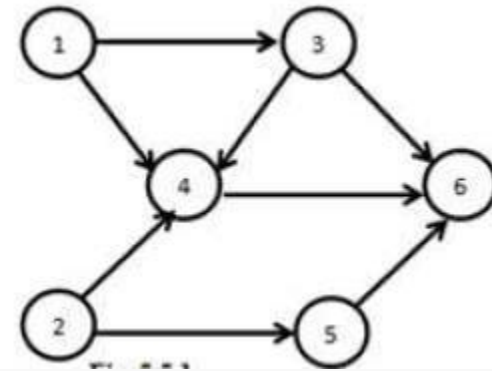
Step 12

enqueue() and dequeue() the vertex 6.

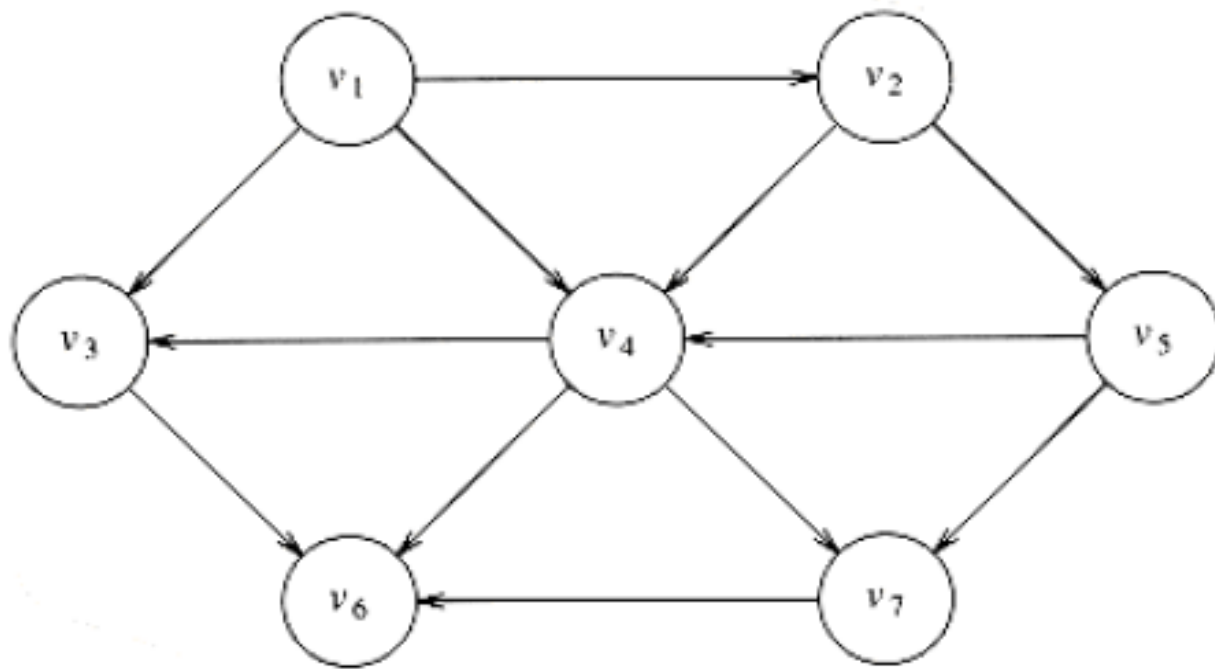
The topological sorted order is

1 2 3 5 4 6

Vertices	Indegree					
1	0					
2	0					
3	1	0				
4	3	2	1	0		
5	1		0			
6	3			2	1	0
enqueue()	1,2	3	5	4		6
dequeue()	1	2	3	5	4	6



TOPOLOGICAL SORTING EX-2



Indegree Before Dequeue #

Vertex	1	2	3	4	5	6	7
v_1	0	0	0	0	0	0	0
v_2	1	0	0	0	0	0	0
v_3	2	1	1	1	0	0	0
v_4	3	2	1	0	0	0	0
v_5	1	1	0	0	0	0	0
v_6	3	3	3	3	2	1	0
v_7	2	2	2	1	0	0	0

Applications:

- Topological Sorting is mainly used for scheduling jobs from the given dependencies among jobs.
- The jobs are represented by vertices, and there is an edge from x to y if job x must be completed before job y can be started
- For example, in constructing a building, the basement must be completed before the first floor, which must be completed before the second floor and so on.
- A topological sort gives an order in which we should perform the jobs.
- In computer science, applications of this type arise in instruction scheduling, ordering of formula cell evaluation when recomputing formula values in spreadsheets,
- Determining the order of compilation tasks to perform in make files
- Data Serialization

Difference Between Tree and Graph

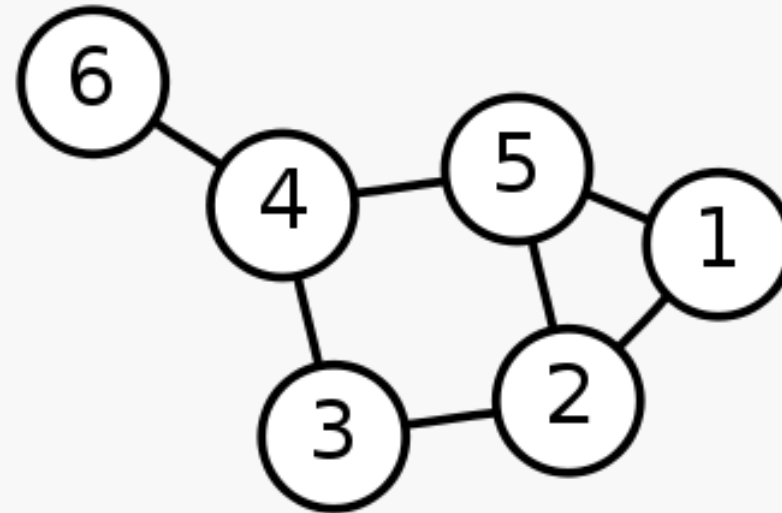
TREE	GRAPH
A tree is a special kind of graph that there are never multiple paths exist. There is always one way to get from A to B.	In graph there can be more than one path i.e graph can have uni-directional or bi-directional path (edges) between nodes.
Pre-order, in-order, and post-order are some kind of the logarithms that are used in trees to through all elements.	Breath First Search, Depth First Search are some kind of searching algorithms in graphs to traverse through each element.
There is a unique node called root in trees.	There is no unique node called root in graph
Tree is a hierarchical model structure.	Graph is network model.

CONT..

All trees are graphs.	But all graphs are not trees.
Main use of trees is for sorting and traversing.	Main use of graphs is coloring and job scheduling.
Less in complexity compared to graphs.	High complexity than trees due to loops.
Trees are directed acyclic graphs.	Graphs are cyclic or acyclic.
Tree contains no loops, no circuits.	Graph may contain self-loops, loops.
Tree must be connected.	Graph may not be connected.

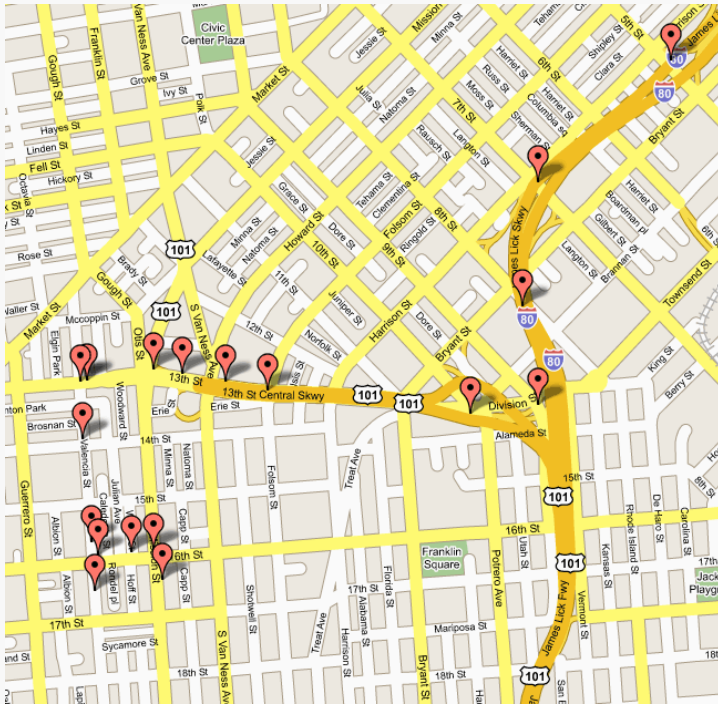
Single-Source Shortest Path Problem

Single-Source Shortest Path Problem - The problem of finding shortest paths from a source vertex v to all other vertices in the graph.



Applications

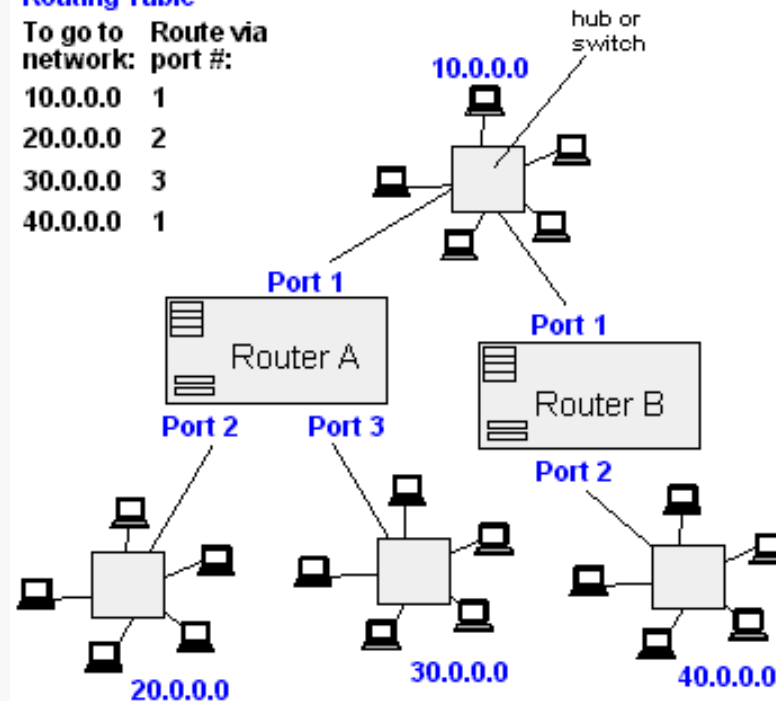
- Maps (Map Quest, Google Maps)
- Routing Systems



From Computer Desktop Encyclopedia
© 1998 The Computer Language Co. Inc.

Router A Routing Table

To go to network:	Route via port #:
10.0.0.0	1
20.0.0.0	2
30.0.0.0	3
40.0.0.0	1



Dijkstra's algorithm

Dijkstra's algorithm - is a solution to the single-source shortest path problem in graph theory.

Works on both directed and undirected graphs. However, all edges must have nonnegative weights.

Input: Weighted graph $G=\{E,V\}$ and source vertex $v \in V$, such that all edge weights are nonnegative

Output: Lengths of shortest paths (or the shortest paths themselves) from a given source vertex $v \in V$ to all other vertices

■ Basics of Dijkstra's Algorithm

- Dijkstra's Algorithm basically starts at the node that you choose (the source node) and it analyzes the graph to find the shortest path between that node and all the other nodes in the graph.
 - The algorithm keeps track of the currently known shortest distance from each node to the source node and it updates these values if it finds a shorter path.
 - Once the algorithm has found the shortest path between the source node and another node, that node is marked as "visited" and added to the path.
 - The process continues until all the nodes in the graph have been added to the path. This way, we have a path that connects the source node to all other nodes following the shortest path possible to reach each node.
- This thirty-year-old solution is a prime example of a greedy algorithm. Greedy algorithms generally solve a problem in stages by doing what appears to be the best thing at each stage.

Approach

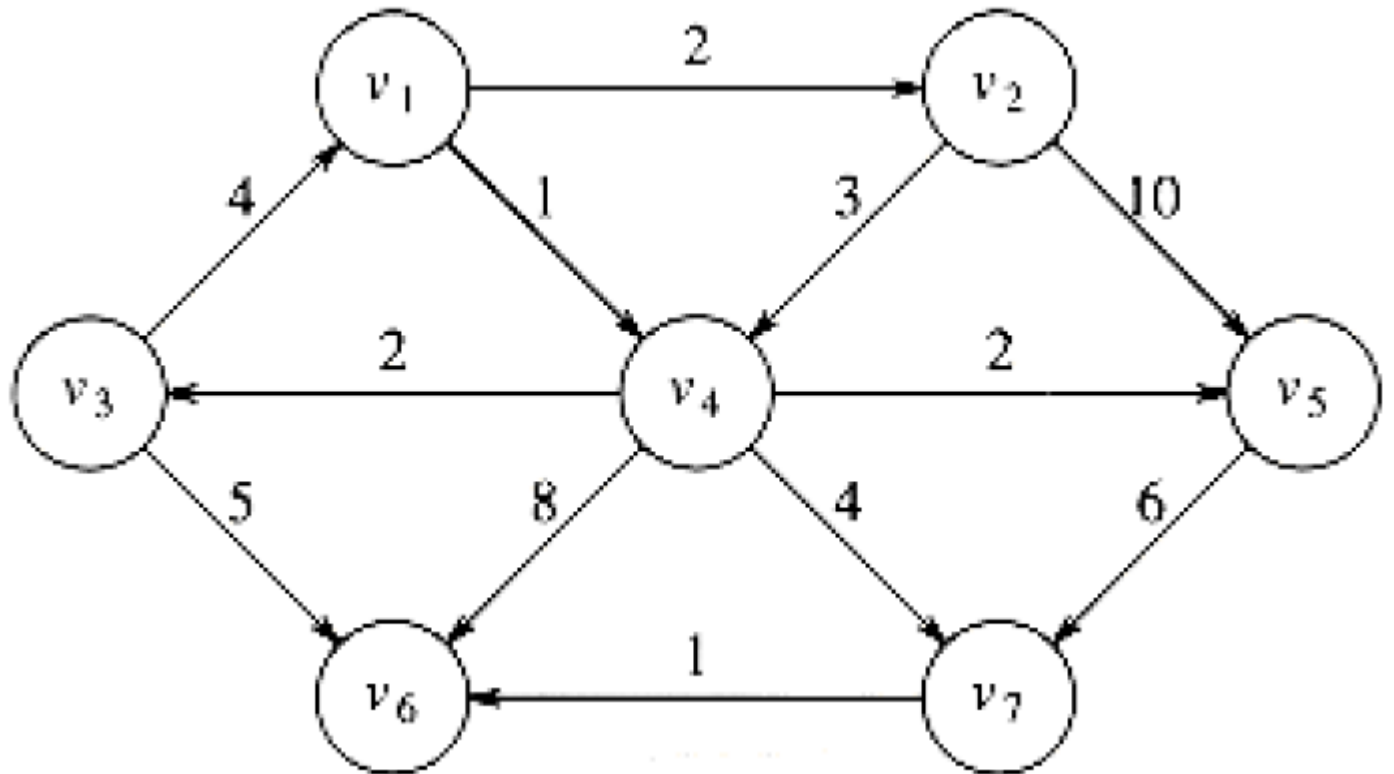
1. Set the distance to the source to 0 and the distance to the remaining vertices to infinity.
2. Set the **current** vertex to the source.
3. Flag the **current** vertex as visited.
4. For all vertices adjacent to the **current** vertex, set the distance from the source to the **adjacent** vertex equal to the minimum of its present distance and the **sum** of the **weight of the edge** from the current vertex to the adjacent vertex and the distance from the source to the **current** vertex.
5. From the set of **unvisited vertices**, arbitrarily set one as the new **current** vertex, provided that there exists an edge to it such that it is the minimum of all edges from a vertex in the set of **visited vertices** to a vertex in the set of **unvisited vertices**. To reiterate: The new current vertex must be unvisited and have a minimum weight edges from a visited vertex to it. This can be done trivially by looping through all visited vertices and all adjacent unvisited vertices to those visited vertices, keeping the vertex with the minimum weight edge connecting it.
6. Repeat steps 3-5 until all vertices are flagged as visited

Pseudocode for Dijkstra's algorithm

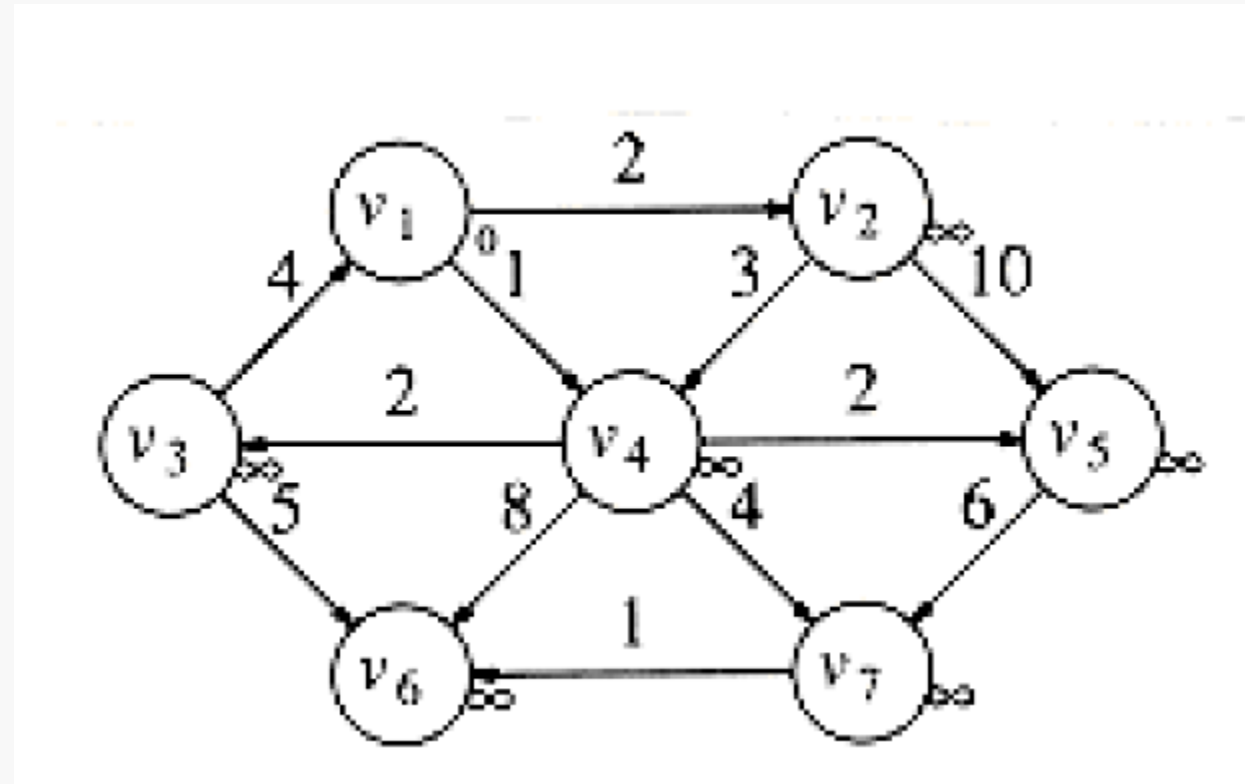
```
void Dijkstra (Vertex s)
{
    PriorityQueue <Vertex> Q = new PriorityQueue <>;
    for each Vertex v except s { v.dist = INFNT; Q.insert(v); }
    s.dist = 0; s.prev= null;
    Q.insert(s);
    while ( !Q.isEmpty() )
    {
        v = Q.deleteMin();
        for each vertex w adjacent to v
        {
            if (w.dist > v.dist + weight of edge (v,w)) {
                w.dist = v.dist + weight of edge (v,w);
                w.prev= v;
                Q.updatePriority(w, v.dist + weight of edge (v,w));
            }
        }
    }
}
```

EXAMPLE 1

- Use this graph to find the shortest path to all the nodes
- At each stage, Dijkstra's algorithm selects a vertex v , which has the smallest d_v among all the unknown vertices, and declares that the shortest path from s to v is known. The remainder of a stage consists of updating the values of d_w .
- we should set $d_w = d_v + c_{v,w}$ if this new value for d_w would be an improvement

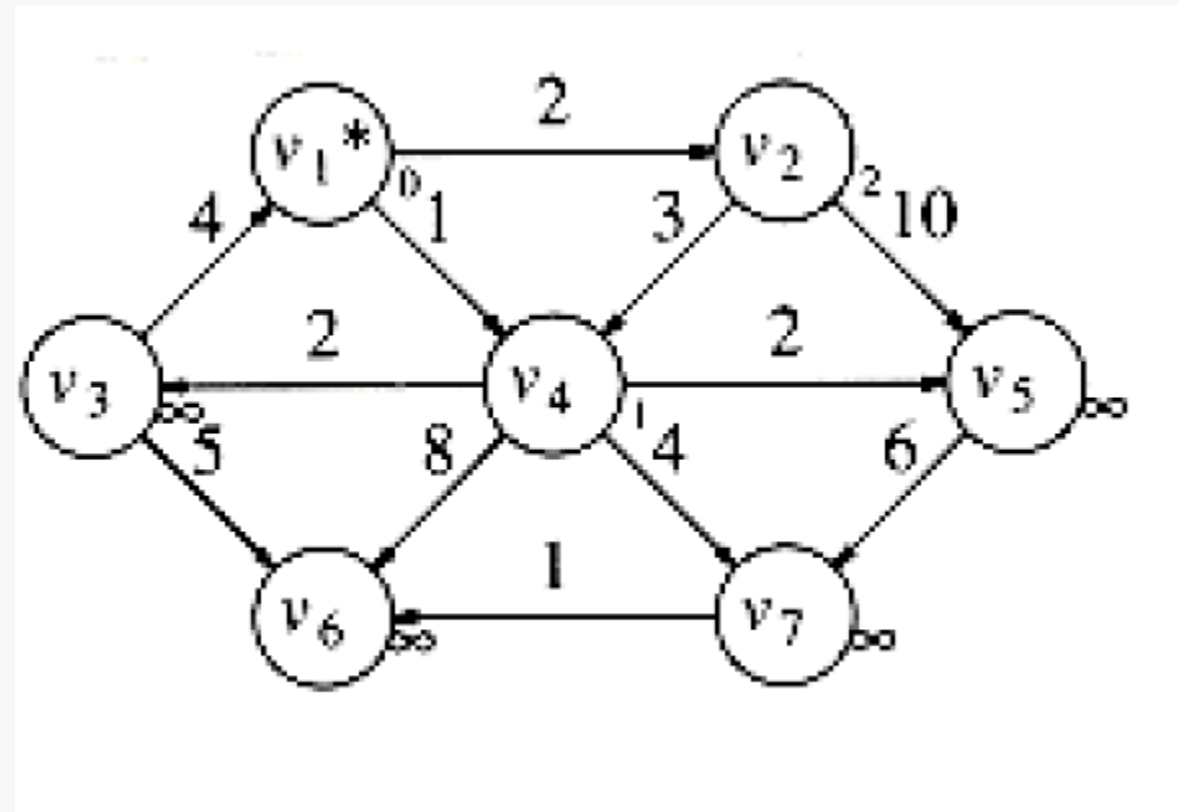


v	Known	d_v	p_v
v_1	0	0	0
v_2	0	∞	0
v_3	0	∞	0
v_4	0	∞	0
v_5	0	∞	0
v_6	0	∞	0
v_7	0	∞	0



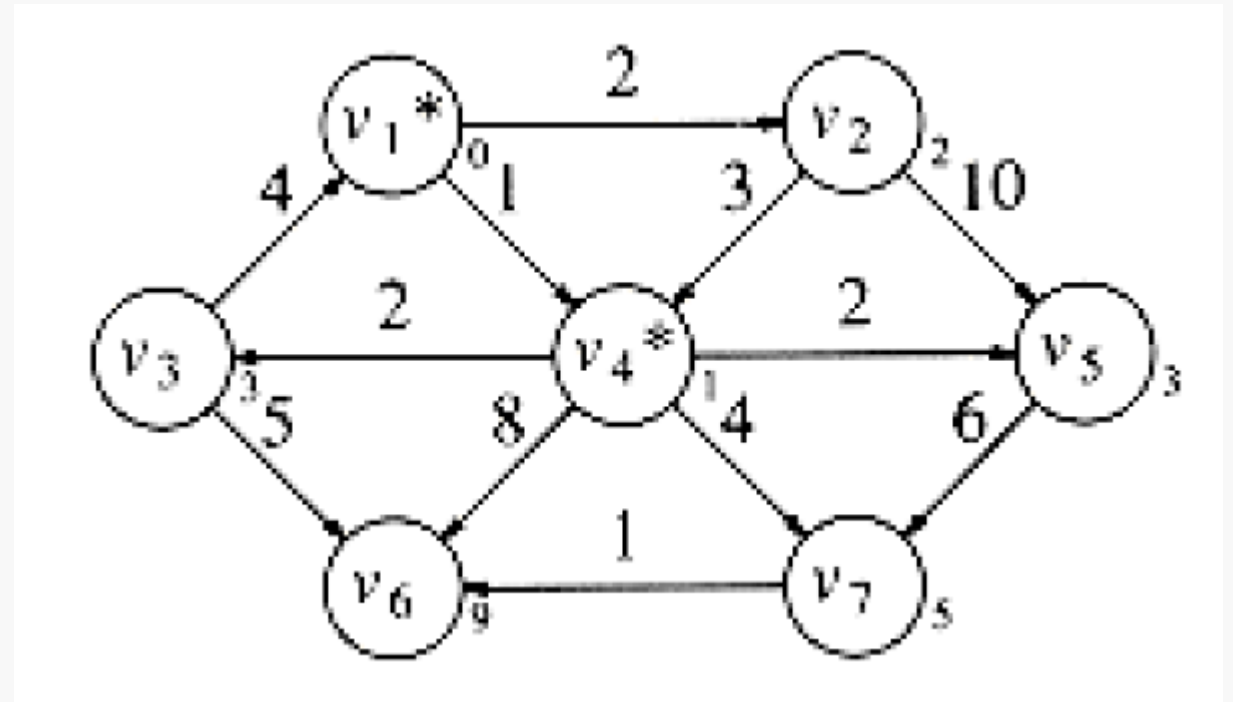
the initial configuration of the table assuming that the start node, s , is v_1 . The first vertex selected is v_1 , with path length 0

v	Known	d_v	p_v
v_1	1	0	0
v_2	0	2	v_1
v_3	0	∞	0
v_4	0	1	v_1
v_5	0	∞	0
v_6	0	∞	0
v_7	0	∞	0



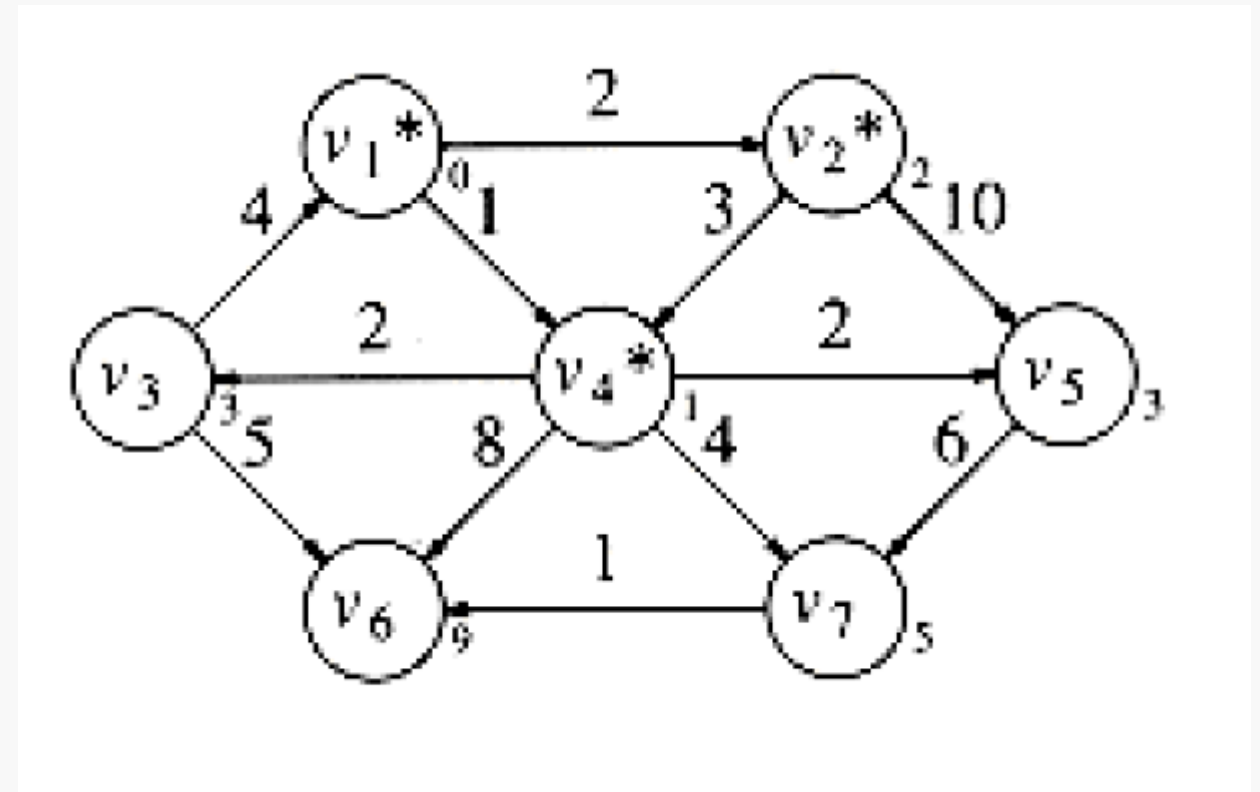
Now v_1 is marked as known, so some entries need to be adjusted. The vertices adjacent to v_1 are v_2 and v_4 . Both these vertices get their entries adjusted, as indicated in Figure

v	Known	d_v	p_v
v_1	1	0	0
v_2	0	2	v_1
v_3	0	3	v_4
v_4	1	1	v_1
v_5	0	3	v_4
v_6	0	9	v_4
v_7	0	5	v_4



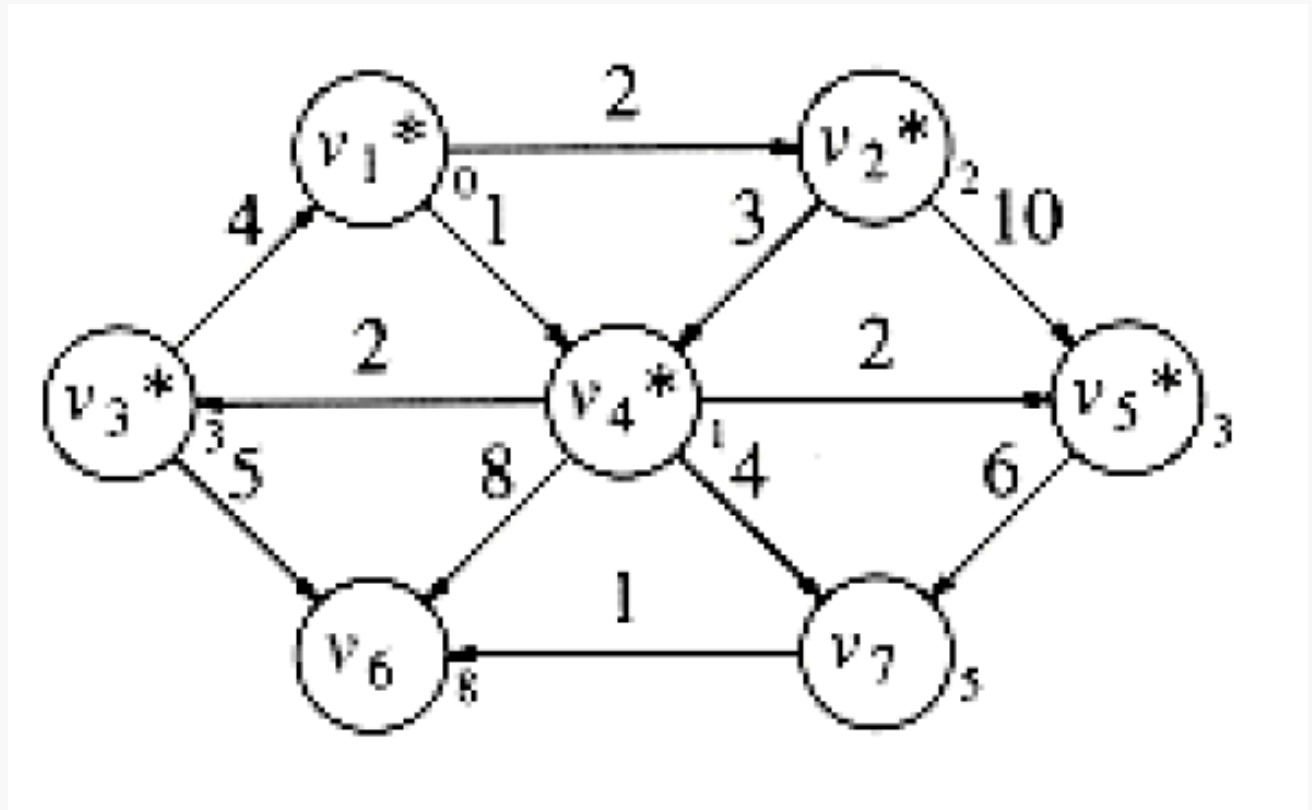
Next v_4 is selected as it has the minimum value in d_v from the list of unknown **vertex** and marked known. Vertices v_3 , v_5 , v_6 , and v_7 are adjacent, and it turns out that all require adjusting, as shown in Figure

v	Known	d_v	p_v
v_1	1	0	0
v_2	0	2	v_1
v_3	0	3	v_4
v_4	1	1	v_1
v_5	0	3	v_4
v_6	0	9	v_4
v_7	0	5	v_4



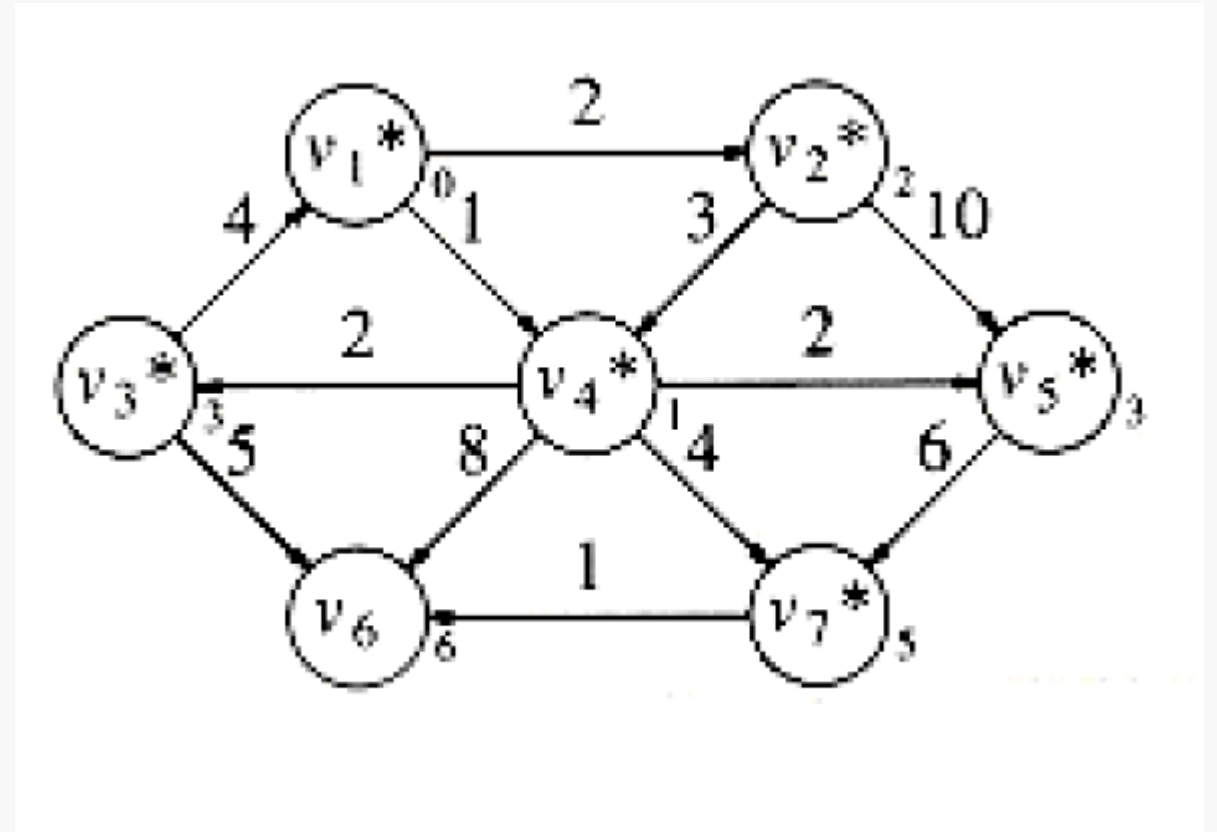
Next v_2 is selected as it has the minimum value in d_v from the list of unknown vertex.. v_4 is adjacent but already known, so no work is performed on it. v_5 is adjacent but not adjusted, because the cost of going through v_2 is $2 + 10 = 12$ and a path of length 3 is already known and it is less than 12.

v	Known	d_v	p_v
v_1	1	0	0
v_2	1	2	v_1
v_3	1	3	v_4
v_4	1	1	v_1
v_5	1	3	v_4
v_6	0	8	v_3
v_7	0	5	v_4



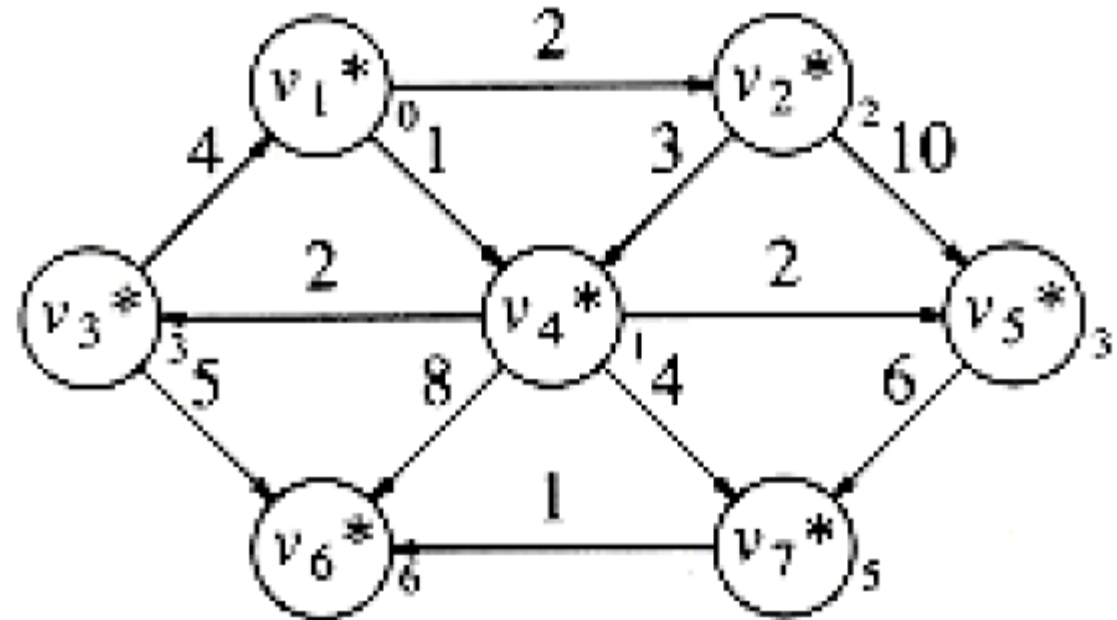
Now v_5 and then v_3 are declared known because both have the minimum value in d_v from the list of unknown vertex. The next vertex selected is v_5 at cost 3. v_7 is the only adjacent vertex, but it is not adjusted, because $3 + 6 > 5$. Then v_3 is selected, and the distance for v_6 is adjusted down to $3 + 5 = 8$.

v	Known	d_v	p_v
v_1	1	0	0
v_2	1	2	v_1
v_3	1	3	v_4
v_4	1	1	v_1
v_5	1	3	v_4
v_6	0	6	v_7
v_7	1	5	v_4



Next v_7 is selected as it has the minimum value in d_v from the list of unknown vertex.;
 v_6 gets updated down to $5 + 1 = 6$.

v	Known	d_v	p_v
v_1	1	0	0
v_2	1	2	v_1
v_3	1	3	v_4
v_4	1	1	v_1
v_5	1	3	v_4
v_6	1	6	v_7
v_7	1	5	v_4



- Finally, v_6 is selected.

References for Dijkstra's Algorithm

<https://www.youtube.com/watch?v=XB4MIexjvY0>

<https://www.cs.usfca.edu/~galles/visualization/Dijkstra.html>

<http://mat.uab.cat/~alseda/MasterOpt/MyL09.pdf>

<https://www.freecodecamp.org/news/dijkstras-shortest-path-algorithm-visual-introduction/>

Minimum Spanning Tree

- Given an undirected weighted graph, a minimum spanning tree (MST) is a subgraph that connects all the vertices with the lowest possible sum of its edge weights.

PROPERTIES

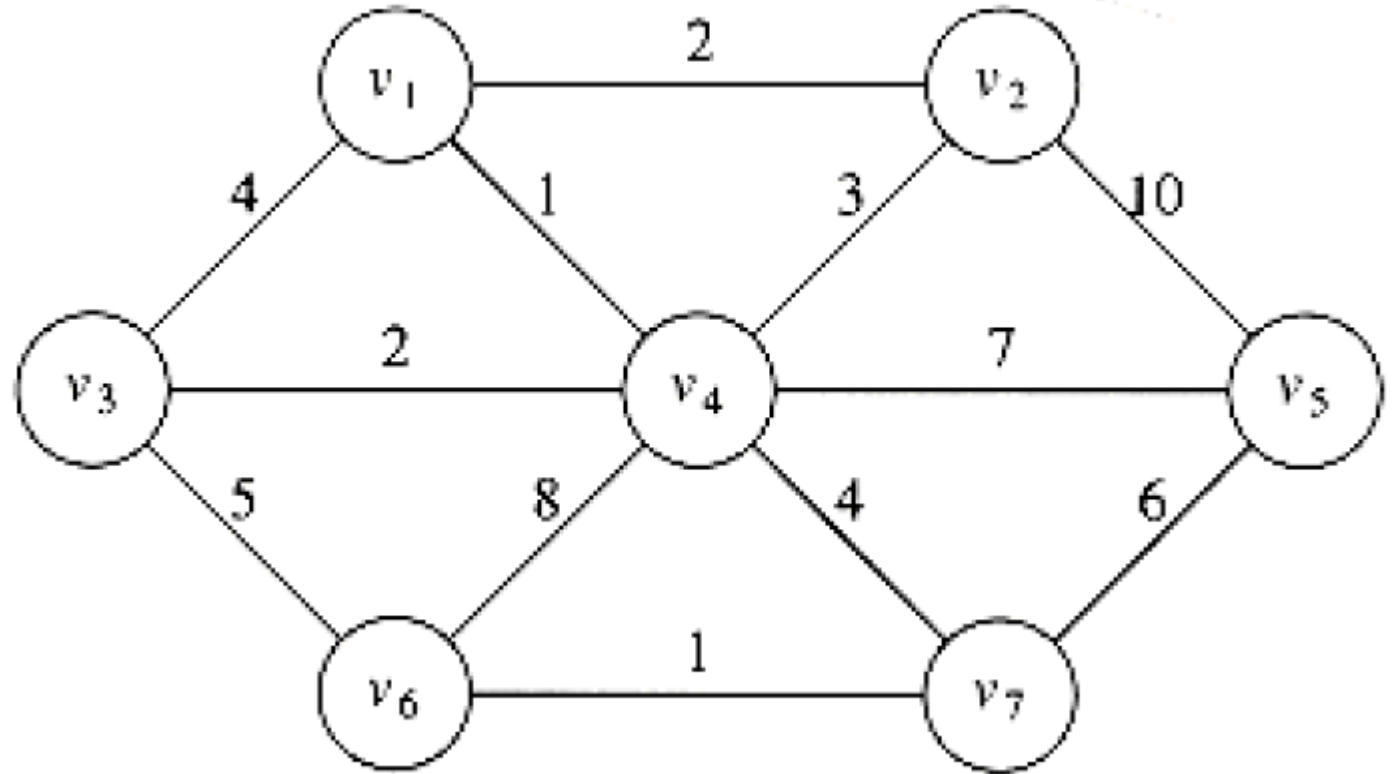
- A minimum spanning tree exists if and only if G is connected.
- number of edges in the minimum spanning tree is $|V| - 1$. (V is the number of vertices)
- A disconnected graph does not have any spanning tree, as it cannot be spanned to all its vertices.
- The spanning tree does not have any cycle (loops)
- A connected graph G can have more than one spanning tree.

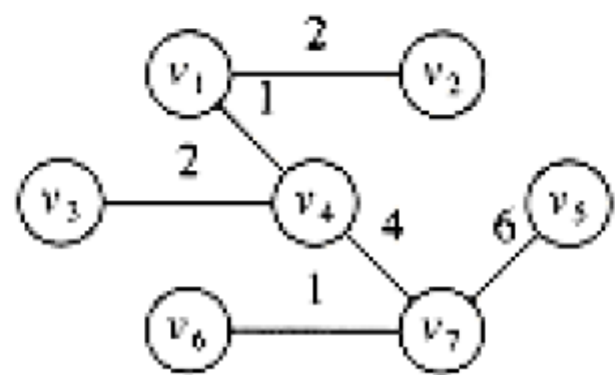
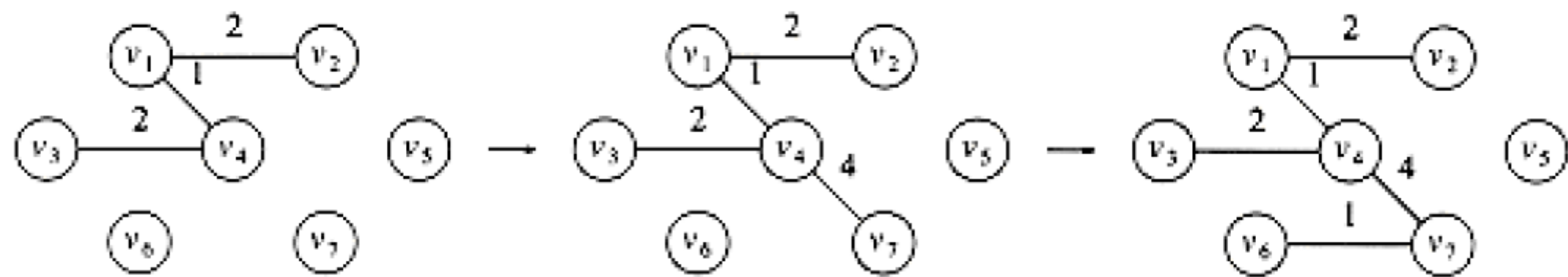
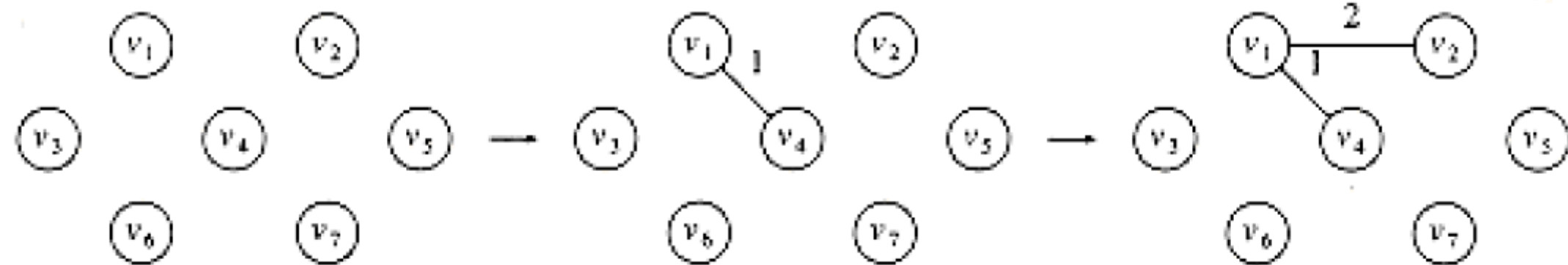
Prim's Algorithm

- It grows the tree in successive stages.
- In each stage, one node is picked as the root, and we add an edge, and thus an associated vertex, to the tree.
- At any point in the algorithm, we can see that we have a set of vertices that have already been included in the tree; the rest of the vertices have not.
- The algorithm then finds, at each stage, a new vertex to add to the tree by choosing the edge (u, v) such that the cost of (u, v) is the smallest among all edges where u is in the tree and v is not.
- Uses Priority Queue.

Example

- Consider the algorithm would build the minimum spanning tree, starting from v_1 .
- Initially, v_1 is in the tree as a root with no edges. Each step adds one edge and one vertex to the tree.
- for each vertex we keep values d_v and p_v and an indication of whether it is known or unknown.
- d_v is the weight of the shortest arc connecting v to a known vertex, and p_v , as before, is the last vertex to cause a change in d_v .



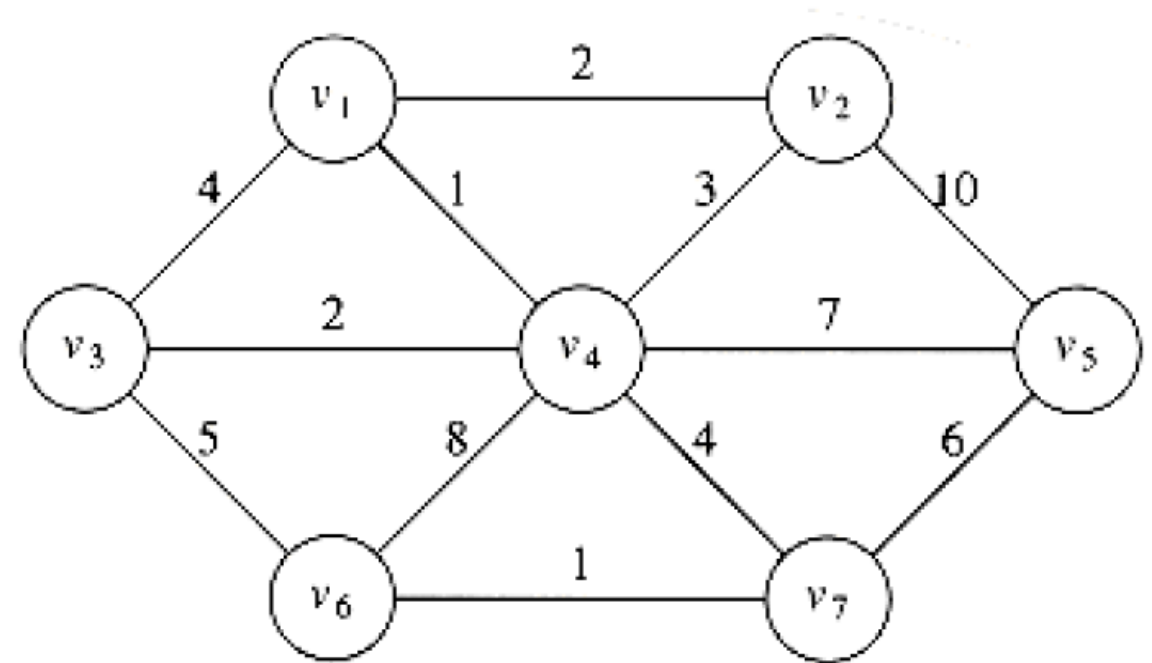


Algorithm for Prim's Minimum Spanning Tree

- **Step 1:** Keep a track of all the vertices that have been visited and added to the spanning tree.
- **Step 2:** Initially the spanning tree is empty.
- **Step 3:** Choose a random **vertex**, and add it to the spanning tree. This becomes the **root node**.
- **Step 4:** Add a new vertex, say **x**, such that
 - **x** is not in the already built spanning tree.
 - **x** is connected to the built spanning tree using minimum weight edge. (Thus, **x** can be adjacent to any of the nodes that have already been added in the spanning tree).
 - Adding **x** to the spanning tree should not form cycles.
- **Step 5:** Repeat the Step 4, till all the vertices of the graph are added to the spanning tree.
- **Step 6:** Print the total cost of the spanning tree.

v	Known	d_v	p_v
v_1	0	0	0
v_2	0	∞	0
v_3	0	∞	0
v_4	0	∞	0
v_5	0	∞	0
v_6	0	∞	0
v_7	0	∞	0

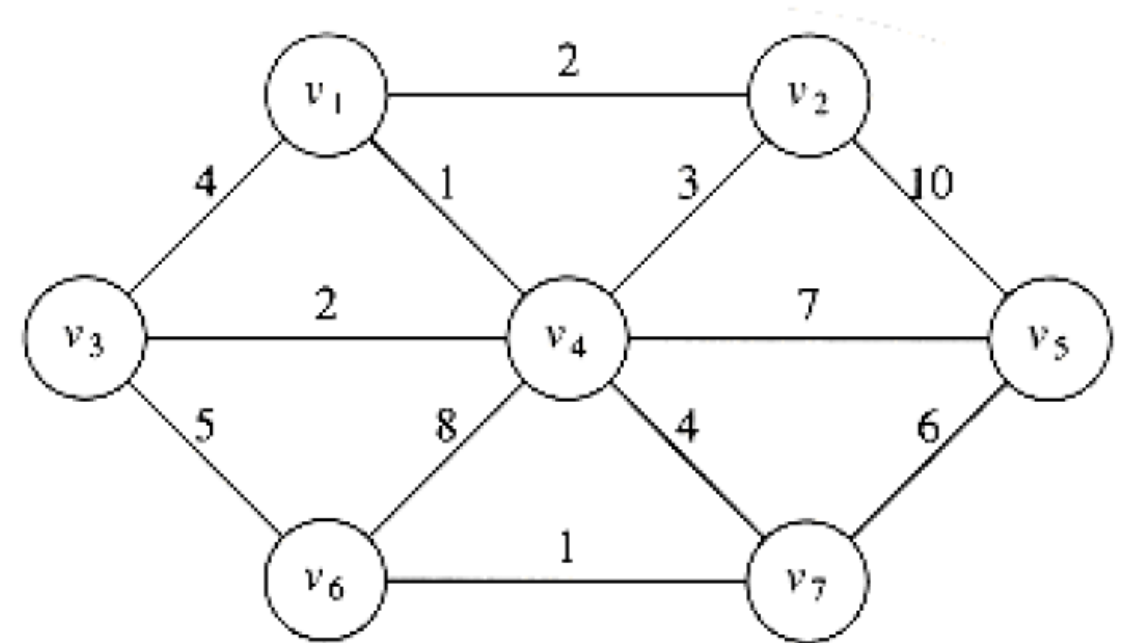
v	Known	d_v	p_v
v_1	0	0	0
v_2	0	2	v_1
v_3	0	4	v_1
v_4	0	1	v_1
v_5	0	∞	0
v_6	0	∞	0
v_7	0	∞	0



- The first table is the initial configuration in here the node v_1 is chosen as the starting root . The second table is updated by including all the edges of v_1 .

v	Known	d_v	p_v
v_1	1	0	0
v_2	0	2	v_1
v_3	0	2	v_4
v_4	1	1	v_1
v_5	0	7	v_4
v_6	0	8	v_4
v_7	0	4	v_4

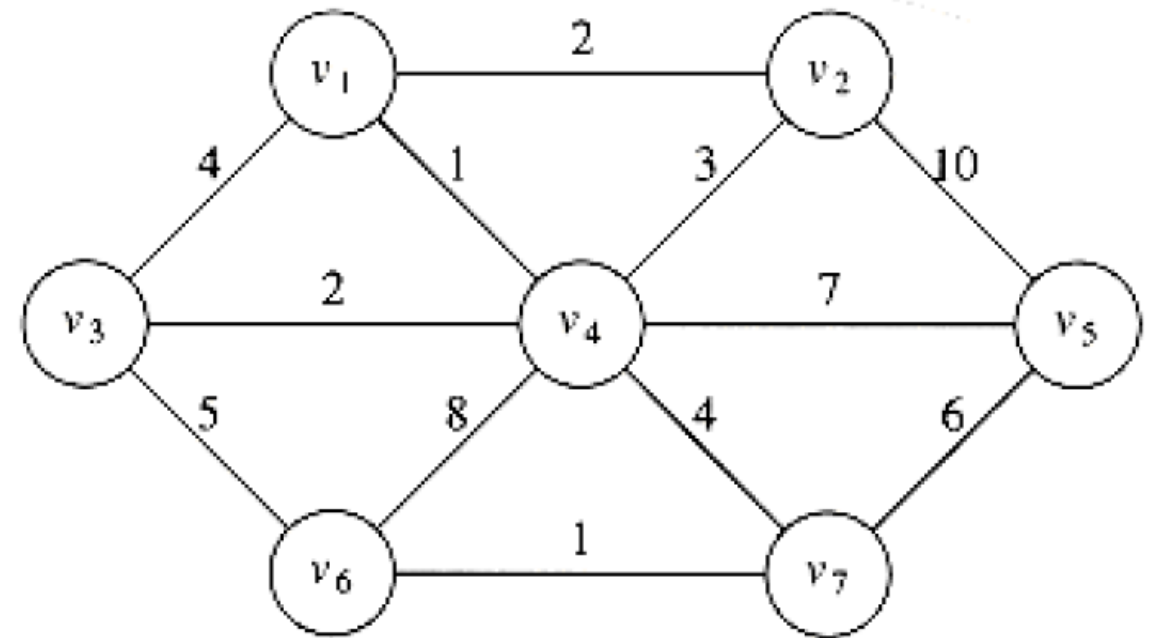
v	Known	d_v	p_v
v_1	1	0	0
v_2	1	2	v_1
v_3	1	2	v_4
v_4	1	1	v_1
v_5	0	7	v_4
v_6	0	5	v_3
v_7	0	4	v_4



- Now among the rest of the edges connecting from either v_1 the minimum least edge among the graph is (v_1, v_4) so v_4 is made known and the adjustment is made in the adjacent nodes of the v_4, v_3 with v_4 as prev edge has a lower cost than v_1 as prev edge so that is updated.
- For updating the next table the least edge from the vertex of v_4 or v_1 is chosen which is (v_4, v_3) and (v_1, v_2) . So both v_2 and v_3 are made known in the second table and the adjustments are made accordingly.
- In the next step v_7 will be made known as it has the lowest value.

v	Known	d_v	p_v
v_1	1	0	0
v_2	1	2	v_1
v_3	1	2	v_4
v_4	1	1	v_1
v_5	0	6	v_7
v_6	0	1	v_7
v_7	1	4	v_4

v	Known	d_v	p_v
v_1	1	0	0
v_2	1	2	v_1
v_3	1	2	v_4
v_4	1	1	v_1
v_5	1	6	v_7
v_6	1	1	v_7
v_7	1	4	v_4



- When v_7 is made known v_5 and v_6 are adjusted
- In the next table v_6 is first made known which doesn't alter any value and then next v_5 is made known

Video reference for Prims

- <https://www.youtube.com/watch?v=4ZlRH0eK-qQ&t=44s>
- <https://www.youtube.com/watch?v=5M7b0Xrn54A>

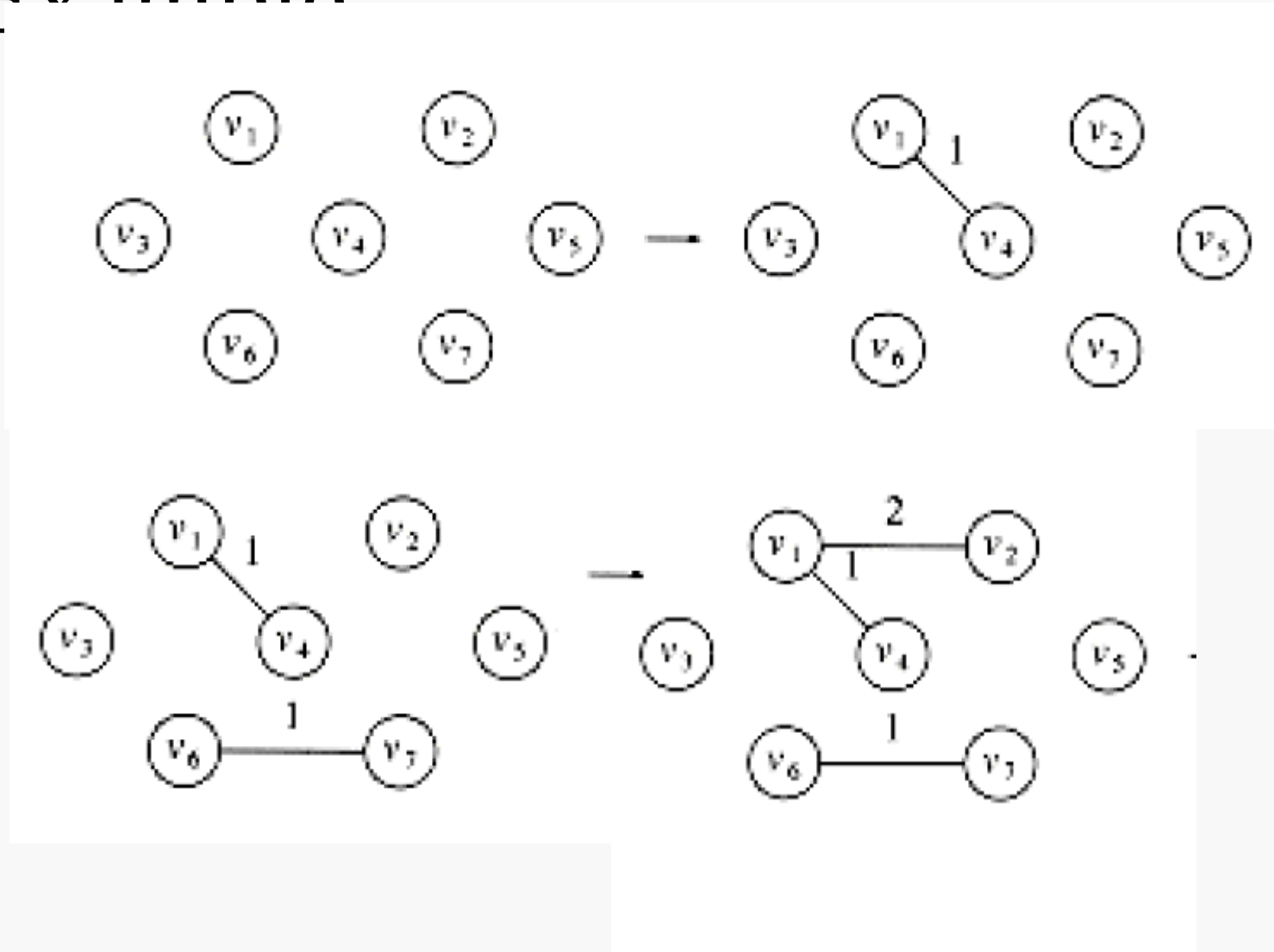
Kruskal's Algorithm

- It falls under a class of algorithms called greedy algorithms that find the local optimum in the hopes of finding a global optimum.
- greedy strategy is continually to select the edges in order of smallest weight and accept an edge if it does not cause a cycle
- Uses Heap

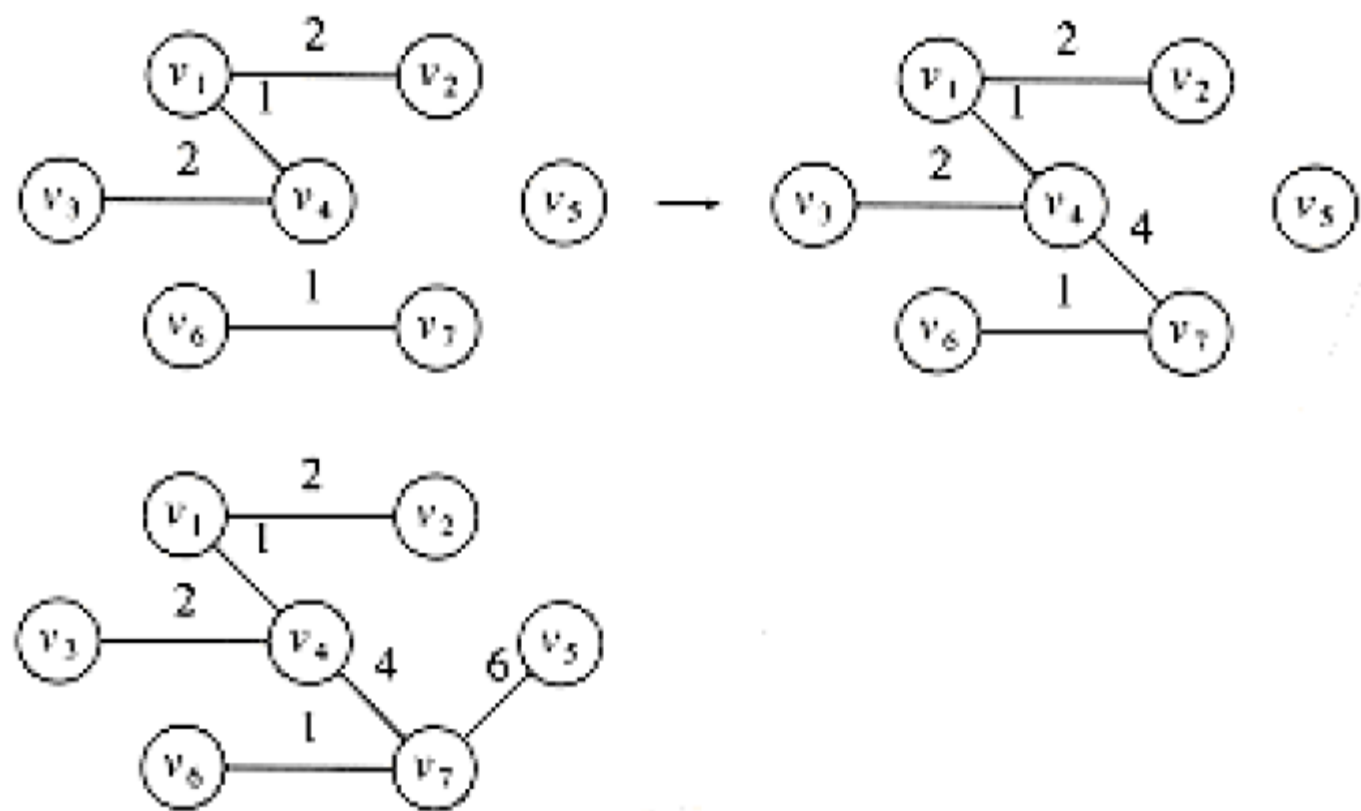
Approach

- The steps for implementing Kruskal's algorithm are as follows:
 1. Sort all the edges from low weight to high
 2. Take the edge with the lowest weight and add it to the spanning tree. If adding the edge created a cycle, then reject this edge.
 - *adding a edge (u,v) is done as follows If u and v are in the same set, the edge is rejected, because since they are already connected, adding (u, v) would form a cycle. As two vertices belong to the same set if and only if they are connected in the current spanning forest.*
 3. Keep adding edges until we reach all vertices.

Example



Edge	Weight	Action
(v_1, v_4)	1	Accepted
(v_6, v_7)	1	Accepted
(v_1, v_2)	2	Accepted
(v_3, v_4)	2	Accepted
(v_2, v_4)	3	Rejected
(v_1, v_3)	4	Rejected
(v_4, v_7)	4	Accepted
(v_3, v_6)	5	Rejected
(v_5, v_7)	6	Accepted



Edge	Weight	Action
------	--------	--------

(v_1, v_4)	1	Accepted
--------------	---	----------

(v_6, v_7)	1	Accepted
--------------	---	----------

(v_1, v_2)	2	Accepted
--------------	---	----------

(v_3, v_4)	2	Accepted
--------------	---	----------

(v_2, v_4)	3	Rejected
--------------	---	----------

(v_1, v_3)	4	Rejected
--------------	---	----------

(v_4, v_7)	4	Accepted
--------------	---	----------

(v_3, v_6)	5	Rejected
--------------	---	----------

(v_5, v_7)	6	Accepted
--------------	---	----------

Reference

- <https://www.javatpoint.com/kruskals-minimum-spanning-tree-algorithm>